Pontificia Universidad Javeriana Cali
Facultad de Ingeniería.
Ingeniería de Sistemas y Computación.
Proyecto de Grado.

Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente

Julio Alejandro Mazo Reyes

Director: MSc. Mario Julián Mora Cardona

27/05/2026



Santiago de Cali, 27/05/2026.

Señores
Pontificia Universidad Javeriana Cali.
Dr. Gerardo Mauricio Sarria
Director Carrera de Ingeniería de Sistemas y Computación.
Cali.

Cordial Saludo.

Por medio de la presente me permito informarle que el estudiante de Ingeniería de Sistemas y Computación Julio Alejandro Mazo Reyes (cod: 8962011) trabaja bajo mi dirección en el proyecto de grado titulado "**Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente**".

Atentamente,

MSc. Mario Julián Mora Cardona

Santiago de Cali, 27/05/2026.

Señores
Pontificia Universidad Javeriana Cali.
Dr. Gerardo Mauricio Sarria
Director Carrera de Ingeniería de Sistemas y Computación.
Cali.

Cordial Saludo.

Me permito presentar a su consideración el anteproyecto de grado titulado "**Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente**" con el fin de cumplir con los requisitos exigidos por la Universidad para llevar a cabo el proyecto de grado y posteriormente optar al título de Ingeniero de Sistemas y Computación.

Al firmar aquí, doy fe que entiendo y conozco las directrices para la presentación de trabajos de grado de la Facultad de Ingeniería aprobadas el 26 de Noviembre de 2009, donde se establecen los plazos y normas para el desarrollo del anteproyecto y del trabajo de grado.

Atentamente,

Julio Alejandro Mazo Reyes
Código: 8962011

Resumen

Este proyecto presenta el desarrollo de un prototipo de plataforma web que integra un Sistema de Tutoría Inteligente (ITS) basado en Modelos de Lenguaje de Gran Escala (LLM), orientado a la identificación de falencias en el aprendizaje y la generación de retroalimentación en cursos de Programación. El sistema es configurado mediante técnicas de prompting para guiar al estudiante a través del método socrático, y ofrece al docente un panel de visualización con métricas sobre el proceso de aprendizaje del grupo. La plataforma es desarrollada en Java Spring Boot con Supabase como Base de datos, documentada bajo el modelo C4 y evaluada mediante encuestas de percepción basadas en los modelos SUS y TAM.

Palabras Clave: Sistema Inteligente de Tutoría(ITS), Inteligencia Artificial Generativa, Prompting, C4, Retroalimentación, Programación, Método Socrático.

Índice general

1. Descripción del Problema **11**
1.1. Planteamiento del Problema . . . 11
1.1.1. Formulación . . . 12
1.1.2. Sistematización . . . 12
1.2. Objetivos . . . 13
1.2.1. Objetivo General . . . 13
1.2.2. Objetivos Específicos . . . 13
1.3. Justificación . . . 13
1.4. Alcances . . . 14
1.4.1. Delimitaciones y Entregables . . . 15
1.5. Metodología . . . 16
1.5.1. Tipo de Estudio. . . . 16
1.5.2. Actividades . . . 16
1.6. Cronograma . . . 18

2. Marco Teórico y Trabajos Relacionados **19**
2.1. Marco Teórico . . . 19
2.2. Trabajos Relacionados . . . 28
2.3. Diferenciadores del Proyecto . . . 29

3. Desarrollo de la Plataforma **31**
3.1. Revisión de Literatura para la Definición de Tipo de Feedback Requerido . . . 31
3.2. Requerimientos de un Sistema de Tutoría Inteligente . . . 33
3.3. Definición de Requerimientos, Métricas, Módulos y Tecnologías de la Plataforma . . . 34
3.3.1. Tecnologías de la Plataforma . . . 34
3.3.2. Módulos de la Plataforma . . . 35
3.3.3. Requisitos Funcionales . . . 36
3.3.4. Requisitos No Funcionales . . . 38
3.3.5. Métricas de Seguimiento . . . 39
3.4. Decisiones de Diseño de la Arquitectura de Prompts y Persistencia . . . 40
3.4.1. Arquitectura de Prompts y Persistencia de Datos . . . 40
3.4.2. Definicion de API de IA a Utilizar . . . 44
3.4.3. Flujos de Datos y Persistencia . . . 46
3.5. Modelo C4 . . . 49
3.5.1. Diseño de Diagrama de Contexto . . . 49
3.5.2. Diagrama de Contenedores . . . 50
3.5.3. Diagrama de Componentes . . . 51

3.5.4. Esquema de Base de Datos 53
3.6. Decisiones de Arquitectura y Desarrollo del Prototipo de la Plataforma 54
3.6.1. Arquitectura y Capa Web: Spring MVC y Spring Boot Starter Web 54

4. Conclusiones y Trabajos Futuros **57**
4.1. Conclusiones 57
4.2. Trabajos Futuros 57

Introducción

En la actualidad, la enseñanza de la programación en la educación superior enfrenta desafíos significativos debido a la alta demanda de estudiantes y la complejidad de conceptos abstractos y altamente lógicos. Esta situación suele aparecer por la falta de capacidad de los docentes para ofrecer un seguimiento individualizado y adaptable al mismo tiempo genera una "brecha de retroalimentación" donde el estudiante no recibe guías oportunas y el profesor carece de visibilidad sobre los obstáculos específicos que enfrenta cada alumno durante su proceso de aprendizaje autónomo.

Como respuesta a esta problemática, los Sistemas de Tutoría Inteligente (ITS) han emergido como herramientas capaces de emular la labor de un tutor humano mediante el uso de Inteligencia Artificial. Sin embargo, muchos de estos sistemas se limitan a entregar respuestas directas, lo que puede reducir el esfuerzo cognitivo del estudiante. Por este motivo, el presente proyecto propone el desarrollo de un prototipo de plataforma web que utiliza un (ITS) de diálogo socrático mediado por IA, que en lugar de resolver el problema por el alumno, el sistema actúa como un guía que formula preguntas y analogías para fomentar el pensamiento crítico.

El proyecto se enfoca en integrar este asistente conversacional con un módulo de métricas para el docente, permitiéndole identificar falencias recurrentes y niveles de dominio de los temas abordados. Para ello, mediante una estructuración modular de la plataforma y el uso de técnicas de prompting, se busca no solo apoyar la labor del profesor, sino también transformar la interacción con la IA en una experiencia pedagógica estructurada que fortalezca las competencias lógicas de los estudiantes de programación.

Capítulo 1

Descripción del Problema

1.1. Planteamiento del Problema

Históricamente, la enseñanza en ciencias formales como la computación ha lidiado con la complejidad de transmitir conceptos abstractos y lógicos que no se pueden ver ni tocar pero que si enfrentan retos pedagógicos que persisten hasta la actualidad. En el presente, esta problemática se intensifica en carreras con un alto componente matemático y lógico, como lo puede ser la ingeniería de sistemas, o las propias ciencias de la computación, donde la abstracción necesaria para dominar la programación crea barreras cognitivas en algunas ocasiones críticas en los estudiantes. Sin embargo, con dificultades los estudiantes siguen luchando con el objetivo de comprender conceptos fundamentales como la lógica, la sintaxis, las estructuras y los tipos de datos (Abiodun & Isong, 2025).

Asimismo, en materias como la Programación Orientada a Objetos (POO), esta situación es especialmente crítica, dado que el alumno se enfrenta a múltiples conceptos en un periodo corto de tiempo. La complejidad no solo radica en la lógica, sino en la necesidad de integrar simultáneamente el paradigma, el lenguaje de programación, el entorno de desarrollo y la metodología. Conceptos pilares como la abstracción, la herencia y el polimorfismo se tornan difíciles de asimilar, creando barreras que los métodos tradicionales de enseñanza no logran derribar fácilmente (Almarales & Fernández, 2019).

Por otra parte, según algunos estudios, las problemáticas mencionadas anteriormente generan una alta tasa de reprobación y una falta de autoconfianza en los estudiantes lo que deriva en un rendimiento académico deficiente (Abiodun & Isong, 2025). Lo que se analiza a menudo, es que este tipo de problemas se genera gracias a que el modelo educativo actual ha dependido casi siempre de una perspectiva de transmisión "simple" de conocimientos en donde se ignora las necesidades individuales de los estudiantes. Esto a su vez crea una "brecha de retroalimentación" (feedback gap), donde el estudiante no logra cerrar la distancia entre su desempeño actual y las metas de aprendizaje deseadas debido a la falta de claridad en las guías o la insuficiencia de recursos personalizados (Evans, 2013).

Si bien estos obstáculos han sido altamente estudiados y documentados desde los inicios de la disciplina (Abiodun & Isong, 2025; Evans, 2013), hoy nos encontramos en un punto de inflexión con la llegada de la Inteligencia Artificial Generativa. Esta tecnología no solo se caracteriza por su facilidad de uso, sino que surge como una oportunidad sin precedentes para transformar la educación tradicional que está basada en un modelo de "talla única" y convertirlo en un modelo dinámico e inclusivo que permite la creación de entornos de aprendizaje adaptativos y personalizados que pueden identificar dónde se encuentran las dificultades específicas de un alumno, ofreciendo estrategias

alternativas y retroalimentación inmediata que el sistema convencional no ha logrado cubrir por sí solo (Rodríguez Chávez, 2021).

De este modo, herramientas como los sistemas inteligentes de tutoria (ITS) impulsados por IA, no solo facilitan la recuperación de información y la generación de ideas, sino que ayudan a los estudiantes a comprender mejor el contenido del estudio al adaptarse a su ritmo individual y generar retroalimentación inmediata al estudiante que no se caracterice por la simple transmisión de conocimiento (Rodríguez Chávez, 2021; Zhou, 2025). Sin embargo, el problema persiste en la falta de integración formal de estas herramientas dentro de las actividades pedagógicas actuales, lo que genera una desconexión entre el potencial tecnológico y la práctica educativa en la actualidad.

Por último, si bien en la literatura se encuentran propuestas que abordan parcialmente esta problemática, la mayoría no logra integrar de forma completa los elementos necesarios para un acompañamiento pedagógico. Sistemas como OATutor, aunque efectivos para el seguimiento del dominio cognitivo mediante algoritmos adaptativos como el Bayesian Knowledge Tracing, dependen de estructuras de conocimiento rígidas y predefinidas que no permiten al docente configurar el comportamiento del tutor según el contexto específico de su curso (Pardos et al., 2023). Por otro lado, plataformas como CodeEdu incorporan agentes LLM especializados con capacidades de tutoría en tiempo real e informes de aprendizaje (Zhao et al., 2025), pero carecen de un módulo de seguimiento para el docente, limitando así la visibilidad del proceso de aprendizaje tanto a nivel individual como grupal. Finalmente, propuestas como Classroom Simulacra se enfocan en la simulación de comportamientos estudiantiles para optimizar estrategias de enseñanza (Xu et al., 2025), pero no ofrecen una herramienta directa de interacción entre el estudiante real y el sistema inteligente, ni un panel de retroalimentación basado en interacciones reales. En la revisión de la literatura realizada no se encontró un sistema que combine simultáneamente un diálogo persistente, una configuración pedagógica flexible por parte del docente y visibilidad del proceso de aprendizaje individual y grupal, lo que evidencia la necesidad del presente proyecto.

1.1.1. Formulación

A partir de los elementos expuestos en la descripción del problema, se plantea la siguiente pregunta de investigación, ¿Cómo desarrollar un prototipo de plataforma web que permita identificar falencias en el aprendizaje y generar retroalimentación mediante un sistema de tutoría inteligente, facilitando el seguimiento de los procesos de aprendizaje de los estudiantes de programación?

1.1.2. Sistematización

Con el fin de descomponer la pregunta de investigación en elementos abordables, se presentan las siguientes subpreguntas que orientan el diseño, desarrollo y evaluación del sistema:

1. ¿Qué requisitos funcionales y métricas de uso son necesarios para desarrollar una plataforma web que permita a los docentes realizar el seguimiento al aprendizaje y al comportamiento de los estudiantes durante el proceso de aprendizaje?

2. ¿Cómo diseñar la plataforma web mediante la estructuración de sus módulos?

3. ¿Cómo implementar una plataforma web que integre un sistema de tutoría inteligente (ITS) capaz de guiar la enseñanza de conceptos de programación y gestionar hilos de conversación persistentes para adaptar el progreso educativo al ritmo del estudiante?

4. ¿Cómo evaluar la usabilidad y aceptación del prototipo mediante encuestas de percepción aplicadas a sus usuarios?

1.2. Objetivos

1.2.1. Objetivo General

- Desarrollar un prototipo de plataforma web para la identificación de falencias en el aprendizaje y generación de retroalimentación mediante un sistema de tutoría inteligente socrático, que facilite el seguimiento del proceso de aprendizaje de los estudiantes de programación por parte de los docentes.

1.2.2. Objetivos Específicos

- Identificar los requisitos funcionales y las métricas de uso de la plataforma que permitan a los docentes el seguimiento al aprendizaje de los estudiantes de programación y su comportamiento durante el proceso de aprendizaje.
- Diseñar la plataforma web, mediante la estructuración de sus módulos.
- Implementar el prototipo de la plataforma web con un sistema de tutoría inteligente socrático basado en IA, configurado mediante técnicas de prompting, para el apoyo de las funciones de seguimiento y generación de retroalimentación pedagógica.
- Evaluar la usabilidad y aceptación del prototipo mediante la realización de encuestas de percepción sobre la plataforma.

1.3. Justificación

La creciente demanda de formación en lógica y fundamentos de programación en la educación superior ha puesto en manifiesto la necesidad de herramientas que trasciendan de la simple automatización de respuestas. En carreras de ingeniería, donde la abstracción es una barrera crítica, los sistemas inteligentes de tutoría han demostrado un impacto positivo al ofrecer asistencia adaptada a las necesidades individuales (Rodríguez Chávez, 2021; Zhou, 2025). Sin embargo, la mayoría de las herramientas actuales se limitan a formatos de evaluación cerrados o a la corrección superficial de sintaxis, dejando de lado la construcción profunda del conocimiento.

El presente proyecto se justifica por la necesidad de implementar un modelo de tutoría socrático impulsado por Inteligencia Artificial Generativa. A diferencia de los sistemas tradicionales, esta propuesta no busca entregar soluciones directas, sino guiar al estudiante mediante preguntas y

analogías que fomenten el aprendizaje significativo y que buscan romper la falsa retensión de conocimiento a través de preguntas como lo propone el método socrático (Farieta-Barrera & Murillo, 2023). Al integrar tecnologías modernas como Java Spring Boot y Supabase, el sistema permite gestionar la persistencia del diálogo, permitiendo que la IA mantenga el contexto histórico de las dificultades del alumno y adapte su discurso de manera evolutiva.

Por otra parte, desde una perspectiva técnica, la relevancia de esta investigación radica en el diseño de una plataforma web bajo el modelo C4, que asegure para cada estudiante su retroalimentación. Además, la implementación de este prototipo ofrece una solución a la "brecha de retroalimentación" (feedback gap) en cursos masivos.

Finalmente, este trabajo beneficia tanto a estudiantes como a docentes. Los primeros reciben un acompañamiento personalizado y constante que reduce la frustración y el deficiente desempeño. Los segundos obtienen una herramienta capaz de identificar patrones de error comunes, optimizando la labor pedagógica. Así, el proyecto sienta las bases para el uso de sistemas inteligentes que no solo informan, sino que enseñan a pensar, alineándose con las exigencias de innovación educativa en la formación de futuros ingenieros.

1.4. Alcances

- El presente proyecto contempla el desarrollo de un prototipo funcional de una plataforma web orientada a la identificación de falencias académicas y la generación de retroalimentación, cuya evaluación se realizará en el contexto de temas de Programación Orientada a objetos por su alto componente lógico y abstracto. El sistema integrará un sistema de tutoría inteligente (ITS) basado en Modelos de Lenguaje de Gran Escala (LLM), configurado mediante técnicas de prompting para guiar al estudiante en la construcción de su propio conocimiento y generar reportes útiles para el docente. A continuación se describen los alcances del proyecto:

  - **Requisitos Funcionales y Métricas:** Para dar cumplimiento al primer objetivo, se planea identificar los requisitos funcionales y las métricas de uso de la plataforma. En ese sentido, se planea hacer una revisión de la literatura o una aplicación de encuestas a docentes, lo que permitirá definir las capacidades técnicas necesarias del sistema y los indicadores necesarios para el monitoreo del proceso de aprendizaje de temas de Programación Orientada a Objetos.
  - **Diseño de Plataforma:** Según el segundo objetivo, se procederá con el diseño de la plataforma web mediante la estructuración de sus módulos funcionales utilizando el modelo C4 como marco de visualización de arquitectura. Esta etapa permitirá representar de manera jerárquica la organización de la plataforma, facilitando la comprensión de las interacciones entre los componentes, el flujo de datos y la integración de los servicios de inteligencia artificial.
  - **Implementación del Prototipo:** En relación con el tercer objetivo, se llevará a cabo la construcción del prototipo de la plataforma web, integrando un sistema de tutoría inteligente basado en inteligencia artificial. El núcleo del sistema, o el chat con IA,

se planea configurar mediante técnicas de prompting para lograr un comportamiento socrático, permitiendo así el soporte técnico necesario para las funciones de seguimiento estudiantil y la generación de retroalimentación pedagógica automatizada.

- **Evaluación:** En cumplimiento del cuarto objetivo, se evaluará la usabilidad y aceptación de la plataforma mediante la aplicación de encuestas de percepción basadas en el instrumento de usabilidad SUS (System Usability Scale) y el modelo de aceptación tecnológica TAM, dirigidas a estudiantes de Programación Orientada a Objetos. Los resultados permitirán identificar fortalezas y oportunidades de mejora del prototipo.

1.4.1. Delimitaciones y Entregables

- Para confirmar la viabilidad y el enfoque del proyecto, se establecen las siguientes delimitaciones:

  - **Enfoque de Inteligencia Artificial:** El proyecto no contempla la creación ni el entrenamiento desde cero de un modelo de inteligencia artificial. El "cerebro" del sistema inteligente de tutoría se apoyará en la conexión vía API a modelos LLM comerciales y preentrenados (como GPT o Claude), centrando el esfuerzo en la parametrización de su comportamiento pedagógico por medio de prompting.
  - **Prompts de Prueba:** Con el objetivo de evaluar el funcionamiento del sistema de tutoría inteligente, se entregarán los prompts de prueba utilizados durante la evaluación de la plataforma. Dichos prompts estarán orientados a temas de Programación Orientada a Objetos, que corresponde al contexto académico seleccionado para la evaluación del prototipo, y serán entregados junto con el documento final del proyecto y demás entregables.
  - **Rol del Sistema:** El prototipo se delimita a ser una herramienta de apoyo formativo. No está diseñado para emitir calificaciones definitivas ni para sustituir el criterio evaluativo del docente en la asignatura.
  - **Población y Entorno de Evaluación:** La evaluación del sistema se limitará a pruebas de aceptación y usabilidad en un entorno académico controlado dentro de la Pontificia Universidad Javeriana Cali.
  - **Estructura de la Plataforma:** Se planea entregar la documentación de la plataforma siguiendo los cuatro niveles del modelo C4: diagrama de contexto, de contenedores, de componentes y de código.
  - **Prototipo de Plataforma Web:** Se plantea entregar la aplicación funcional desarrollada en Java Spring Boot y desplegada de manera local con docker. Esta a su vez cual incluye los siguientes módulos: la interfaz de interacción socrática para el estudiante, el panel de visualización y seguimiento para el docente, el panel de configuración de prompts según el tema y la materia, y el panel de ayuda para la construcción de prompts asistida por IA. Adicionalmente, podrán incluirse otros módulos que se consideren necesarios tras el análisis de requisitos.

- **Repositorio de Código Fuente:** El acceso al código organizado se dará en el sistema de control de versiones (GitHub), incluyendo la configuración de la base de datos en Supabase y las funciones de backend.
- **Guía de Gestión Pedagógica para el Docente:** Documento técnico e instructivo orientado al docente, donde se detalla y explica como utilizar la plataforma.

1.5. Metodología

Se ha seleccionado un modelo de desarrollo incremental y evolutivo, el cual permite integrar funcionalidades de manera progresiva y ajustar la lógica del sistema de tutoría inteligente de forma iterativa. Este enfoque asegura la evaluación técnica de cada componente antes de su integración final en el prototipo. Bajo esta premisa, el proyecto se estructura en cuatro fases principales alineadas con el ciclo de vida del diseño de ingeniería:

1. **Análisis de Requisitos:** Identificar los requisitos funcionales y las métricas de uso de la plataforma que permitan a los docentes el seguimiento al aprendizaje de los estudiantes y su comportamiento durante el proceso de aprendizaje.
2. **Diseño de Plataforma:** Diseñar la plataforma web, mediante la estructuración de sus módulos por medio de un marco de visualización de arquitectura.
3. **Desarrollo e Integración de IA:** Implementar el prototipo de la plataforma web con un sistema de tutoría inteligente basado en IA, configurado mediante técnicas de prompting, para el apoyo de las funciones de seguimiento y generación de retroalimentación pedagógica.
4. **Evaluación del Sistema:** Evaluar la usabilidad y aceptación del prototipo mediante la realización de encuestas de percepción sobre la plataforma.

1.5.1. Tipo de Estudio.

El presente trabajo se define como un proyecto aplicado con un enfoque de diseño de ingeniería, centrado en la dimensión teórico-práctica del desarrollo de software. El estudio se orienta a la construcción y ejecución de una solución tecnológica, materializándose en la implementación de un prototipo funcional que integra servicios Inteligencia Artificial. Por su finalidad, se categoriza como investigación aplicada, ya que abarca desde la definición de requisitos técnicos y la ingeniería de prompts hasta la evaluación de la plataforma con encuestas de percepción, consolidando un sistema de información funcional que responde de manera directa a una necesidad educativa específica mediante inteligencia artificial.

1.5.2. Actividades

Se llevaran a cabo las siguientes actividades, alineadas con cada uno de los objetivos específicos del proyecto:

1. **Identificar los requisitos funcionales y las métricas de monitoreo de la plataforma.**
   - 1.1 Identificar los requerimientos funcionales del sistema.
   - 1.2 Determinar las métricas de seguimiento para el monitoreo del uso de la plataforma por medio del docente.
   - 1.3 Establecer los flujos de información para la persistencia de datos entre el front-end el backend y la API.
2. **Definir la estructura de la plataforma bajo el modelo C4 y el diseño de la interfaz.**
   - 2.1 Diseñar diagramas de Contexto y Contenedores para la integración de la IA.
   - 2.2 Elaborar diagramas de Componentes y Código para definir la lógica de la aplicación.
   - 2.3 Diseñar el esquema de la base de datos para el historial y métricas.
   - 2.4 Crear prototipos de interfaz para el chat y el panel del docente.
3. **Implementar el prototipo de la plataforma web con el sistema de tutoría inteligente.**
   - 3.1 Evaluar y elegir la API de IA que mejor se ajuste a los requerimientos técnicos del proyecto.
   - 3.2 Preparar la infraestructura de desarrollo y el entorno de despliegue para el software.
   - 3.3 Verificar que el frontend, el backend y los servicios de la API de IA se comuniquen correctamente antes de iniciar la programación de funciones avanzadas.
   - 3.4 Desarrollar la interfaz para que el profesor configure las instrucciones de la IA según el tema de programación a abordar.
   - 3.5 Construir el panel de visualización y monitoreo de estudiantes, en donde el profesor podrá observar de manera colectiva o individual como van avanzando los estudiantes por medio del uso de la plataforma.
   - 3.6 Construir el panel docente de prompting que ayude al profesor a estructurar el prompt que va a parametrizar el chat con IA para que brinde enseñanza socrática.
   - 3.7 Desarrollar las interfaces y la lógica del servidor necesaria para gestionar las conversaciones entre el estudiante y la IA.
4. **Evaluar el sistema.**
   - 4.1 Aplicar encuestas a estudiantes para medir qué tan fácil es usar la plataforma y qué tan útil la perciben para el aprendizaje.
   - 4.2 Procesar los datos obtenidos en las encuestas para identificar posibles mejoras en el prototipo final.

1.6. Cronograma

Mes	**1**				**2**				**3**				**4**			
Actividades / Semanas	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Fase 1: Requisitos.																
1.1 Identificación de requerimientos																
1.2 Determinación de métricas																
1.3 Flujos de datos y persistencia																
Fase 2: Arquitectura C4.																
2.1 Diseño Contexto y Contenedores																
2.2 Diseño Componentes y Código																
2.3 Esquema de base de datos																
2.4 Prototipado de interfaces																
Fase 3: Implementar el prototipo.																
3.1 Evaluar y elegir la API de IA																
3.2 Preparar infraestructura y entorno de despliegue																
3.3 Verificar conectividad front, back, API																
3.4 Implementación de Panel Prompting Docente																
3.5 Implementación de Panel Monitoreo Docente																
3.6 Implementación de Panel Ayuda Prompting Docente																
3.7 Implementación del Chat Socrático																
Fase 4: Evaluar el sistema.																
4.1 Aplicación de Encuestas de Percepción																
4.2 Análisis de Resultados																

Cuadro 1.1: Cronograma de Actividades del proyecto.

Capítulo 2

Marco Teórico y Trabajos Relacionados

2.1. Marco Teórico

Normatividad

Con el objetivo de orientar las decisiones de seguridad de la información en la plataforma propuesta, se toma como referencia la ISO/IEC 27001, con la cual se establecen algunos requisitos para implementar y mantener un sistema de gestión de seguridad de la información, como lo es la preserva la confidencialidad (International Organization for Standardization, 2022). En ese sentido se busca con la confidencialidad, que el desarrollo y despliegue de la plataforma cumpla con todos los requerimientos de seguridad y que solo el profesor y el estudiante tengan acceso a la información pertinente en los chats.

Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO), es un paradigma de programación que se conoce por un estilo de implementación en el que los programas se organizan como colecciones cooperativas de objetos, siendo cada uno, una representación de una instancia de una clase dentro de una jerarquía unida por relaciones (Almarales & Fernández, 2019). A su vez, este paradigma se fundamenta en principios fundamentales como la abstracción que se sostiene en la capacidad de modelar atributos y relaciones de entidades como clases para definir una representación abstracta de un sistema, el encapsulamiento que permite básicamente ocultar el estado interno y la funcionalidad de un objeto para poder permitir el acceso únicamente a través de un conjunto público de funciones, la herencia que es la capacidad para crear nuevas abstracciones basadas en abstracciones existentes y por último el polimorfismo que es la capacidad para implementar propiedades o métodos heredados de diferentes maneras en múltiples abstracciones (Microsoft Learn, 2026).

Esto desde el punto de vista de programación representa una ventaja importante en temas relacionados como la reducción de código redundante, lo que permite un código conciso y sin repeticiones, la reutilización y extensión de código a través de las clases y por último, permite la facilidad en el desarrollo y mantenimiento debido a la filosofía del paradigma fundamentado en clases (Almarales & Fernández, 2019).

Sistema Inteligente

Este se define como un sistema artificial que opera como un agente y exhibe un comportamiento racional en donde siempre interactua con su entorno mediante la percepción y la acción para realizar tareas de utilidad humana. Su racionalidad se manifiesta de dos formas: actuando racionalmente para maximizar las probabilidades de éxito en sus objetivos y pensando racionalmente al ser capaz de justificar sus conclusiones mediante procesos de razonamiento comprensibles. Estructuralmente, estos sistemas integran funciones de comprensión del entorno, toma de decisiones (ya sea de forma reactiva o deliberada) y comunicación, pudiendo actuar como delegados autónomos que ejecutan tareas por sí mismos o como asesores que proporcionan información experta al usuario. Finalmente, una característica clave es su capacidad de aprendizaje, que les permite mejorar su desempeño con la experiencia o adquirir nuevas creencias mediante la observación y la transferencia de conocimiento (Molina, 2022).

Sistema de Tutoría Inteligente

Un Sistema de Tutoría Inteligente (ITS) es una plataforma de aprendizaje adaptativo fundamentada en las ciencias del aprendizaje que actúa como un tutor personalizado para mejorar el rendimiento académico de los estudiantes. Su característica central y capacidad principal consiste en evaluar de manera continua las habilidades del alumno en componentes de conocimiento específicos para adaptar la instrucción y la práctica según sus necesidades individuales. Esto permite estimar el dominio cognitivo, lo que permite al tutor seleccionar problemas de forma inteligente, evitar la práctica excesiva o insuficiente y proporcionar apoyos pedagógicos inmediatos como pistas (hints) y retroalimentación instantánea sobre la corrección de las respuestas (Rodríguez Chávez, 2021). Es importante entender que el objetivo del proyecto es realizar un (ITS) que a su vez funciona como un sistema inteligente.

Plataformas Educativas

Las plataformas educativas son sistemas en línea diseñados para facilitar la enseñanza y el aprendizaje al ofrecer entornos virtuales con recursos interactivos y diversas actividades. Entre las más relevantes para este proyecto se destacan las Plataformas de Gestión del Aprendizaje (LMS), las cuales permiten administrar cursos y realizar un seguimiento del progreso de los alumnos (Quiñonez et al., 2023). El proyecto como tal adopta este enfoque al permitir que el docente defina las reglas de un curso mediante un prompt, para que luego el sistema guíe al estudiante y retroalimente al profesor.

Por otro lado, existen las plataformas de tutoría en línea, cuyo propósito original es conectar a los alumnos con tutores para recibir un apoyo personalizado (Quiñonez et al., 2023). En el contexto del proyecto, este rol de acompañamiento y guía académica en tiempo real será asumido directamente por el propio sistema de tutoría inteligente, automatizando la orientación personalizada.

Finalmente, también existen las plataformas de aprendizaje adaptativo que emplean algoritmos que recopilan información de los usuarios para ajustar las actividades según sus necesidades indi-

viduales (Quiñonez et al., 2023). Esta lógica se integra a la plataforma propuesta mediante el uso de inteligencia artificial, logrando que el sistema de tutoría inteligente se adapte dinámicamente a las capacidades y requerimientos específicos de cada estudiante.

Retroalimentación

En el contexto de la educación superior, la retroalimentación se identifica como un proceso de intercambio, siempre dinámico que busca reducir la brecha entre el desempeño actual del estudiante y las metas u objetivos ya establecidos para el aprendizaje. Para ello es importante que el estudiante no se considere solo como un receptor, sino un participante activo que hace parte del proceso dinámico y que así mismo se le ofrezcan oportunidades tempranas para recibir comentarios que puedan aplicarse en tareas futuras (Evans, 2013). En este punto, es importante entender que la retroalimentación es parte fundamental de esta investigación, dado que lo que se busca es que el profesor obtenga feedback del proceso que lleva el estudiante con el sistema de tutoría inteligente, para que este a su vez pueda ir estableciendo nuevos objetivos, metas o metodologías que puedan orientar el aprendizaje en los estudiantes.

Modelo C4

El modelo C4 es un marco de trabajo para la abstracción de arquitecturas de software que permite documentar sistemas de manera jerárquica. A diferencia de los diagramas UML tradicionales, este modelo progresa desde una visión general del sistema hacia una perspectiva más técnica, estructurándose en cuatro niveles principales de abstracción. Los dos primeros niveles son el Diagrama de Contexto, que muestra el sistema como una caja negra y su interacción con los usuarios, y el nivel de Contenedores, que desglosa el sistema en aplicaciones, bases de datos, microservicios o sistemas de archivos (Brown, 2022).

Los niveles restantes profundizan en la estructura interna: el nivel de Componentes descompone cada contenedor para mostrar sus módulos internos, tales como controladores o servicios, mientras que el nivel de Código detalla las clases y las diversas herramientas de programación implementadas (Brown, 2022). De esta manera, el modelo ofrece una transición fluida entre la conceptualización de alto nivel y el detalle técnico de la implementación.

Para el desarrollo del proyecto, el modelo C4 se utiliza específicamente para registrar los componentes de la plataforma. Su aplicación permite separar claramente los módulos del sistema de otros elementos clave que forman parte de la infraestructura del desarrollo, como lo es la base de datos implementada en Supabase.

Requisitos Funcionales

Los requisitos funcionales son una parte fundamental de la especificación de un sistema de software. Estos permiten definir las acciones fundamentales que debe realizar el software tanto al aceptar y procesar entradas como al generar las salidas y en ocasiones conviene separar los requisitos en subfunciones o subprocesos (IEEE Computer Society, 1998). Estas acciones fundamentales se expresan generalmente a través de declaraciones imperativas, utilizando la fórmula "el sistema deberá" para cada requisito identificado. Asimismo, estos requisitos deben especificar detalladamente las verificaciones de validez de las entradas, la secuencia exacta de las operaciones y el efecto que tienen los diferentes parámetros en el comportamiento del sistema (IEEE Computer Society, 1998).

Además, la especificación debe dejar clara la relación entre las salidas y las entradas, detallando tanto las secuencias de intercambio de información como las fórmulas necesarias para la conversión de datos (IEEE Computer Society, 1998). Es esencial que estos requisitos se planteen desde un punto de vista puramente externo, enfocándose exclusivamente en los servicios que el software prestará y evitando, por lo general, detalles de diseño interno como la elección de estructuras de datos o la división técnica del software en módulos. Por último, para asegurar la calidad del documento, cada requisito debe ser unívocamente identificable y estar organizado de manera que se maximice su legibilidad para el cliente (IEEE Computer Society, 1998).

API

Una API o "interfaz de programación de aplicaciones" es un mecanismo que permite la comunicación entre dos componentes mediante ciertos protocolo o definiciones. En ese sentido, existen dos términos importantes, cliente y servidor, siendo el cliente el usuario que normalmente solicita o realiza la solicitud al servidor. Entre los tipos de APIs mas comunes están las APIS REST, estas definen un conjunto de funciones como GET, PUT, y DELETE, etc. Por medio de los cuales el cliente y servidor pueden intercambiar solicitudes (Services, s.f.). Este mecanismo es importante, dado que se debe integrar por medio de la API un LLM que funcione como cerebro del sistema de tutoría inteligente que se pretenden entrenar por medio de prompts.

Inteligencia Artificial

La inteligencia artificial es una colección de procesos computacionales orientados a la generación de conocimiento sobre dominios específicos, buscando modelar la inteligencia y las acciones del cerebro humano (Monett & Lewis, 2018). En ese sentido, el texto identifica dos enfoques fundamentales: la IA simbólica, basada en la lógica y el razonamiento, y la IA estadística, la cual analiza grandes cantidades de información para extraer patrones correlacionales mediante mecanismos de aprendizaje automático (Monett & Lewis, 2018). Entre las características más comunes de estos sistemas está su capacidad para adaptarse al entorno y resolver tareas específicas, si bien se argumenta que la IA actual suele carecer de un "aprendizaje real" que implique una comprensión profunda o sentido común sobre la naturaleza de la información que procesa (Monett & Lewis, 2018). Por medio

de estas técnicas de computación epistémica o morfológica, el sistema procesa representaciones de datos para generar respuestas, clasificaciones o predicciones de utilidad práctica (Monett & Lewis, 2018). Este mecanismo tecnológico es de gran importancia para el proyecto, puesto que permite el desarrollo de un Sistema de Tutoría Inteligente capaz de realizar un seguimiento adaptativo que se ajusta en tiempo real a las capacidades y necesidades cambiantes de los estudiantes.

System Usability Scale (SUS)

El SUS es una herramienta de 10 items, creado por John Brooke en 1986 para evaluar la percepción subjetiva de la usabilidad de una interfaz de manera rápida y sin la necesidad de pruebas más sofisticadas. Esta a su vez se destaca por ser una herramienta gratis y de fácil utilización que ofrece resultados confiables sin importar el sistema en el que se aplique o las tareas a evaluar. Por otra parte, su diseño se fundamenta en la norma ISO 9241-11, que define la usabilidad a través de tres pilares principales: la eficacia (logro de la tarea), la eficiencia (rapidez en la ejecución) y la satisfacción (actitud positiva del usuario) (Picolo et al., 2025).

El cuestionario se estructura mediante diez preguntas graduadas en una escala tipo Likert de cinco puntos, que van desde "totalmente en desacuerdo" hasta "totalmente de acuerdo". Estos ítems abarcan diversos criterios esenciales para la calidad del software, evaluando aspectos como la complejidad del sistema, la necesidad de soporte técnico, la integración de sus funciones y la facilidad de aprendizaje de la interfaz. Esta amplitud permite obtener una perspectiva integral del usuario sobre el objeto de estudio de forma sencilla (Picolo et al., 2025). En este proyecto, el SUS permitirá medir si la interacción con el tutor socrático y el panel docente resulta funcional para los usuarios finales.

Technology Acceptance Model (TAM)

Es una teoría que explica por qué las personas deciden usar o no una nueva herramienta digital. Se basa principalmente en dos ideas: la utilidad percibida (el grado en que el usuario cree que el sistema mejorará su desempeño académico o docente), y la facilidad de uso (el grado en que el usuario considera que interactuar con la plataforma no requiere un esfuerzo excesivo). Si una persona siente que la herramienta es útil y sencilla, tendrá una actitud positiva y terminará usándola realmente (González-Bravo & Valdivia-Peralta, 2015).

Por otra parte, el modelo postula que la aceptación de las tecnologías de la información (TIC), se basa en una combinación de tres elementos secuenciales: una actitud positiva hacia la tecnología, la intención de utilizar el sistema y el uso real del mismo. Además, reconoce que en marcos organizacionales el uso de una herramienta no siempre es puramente voluntario, ya que los individuos pueden adoptar una tecnología debido a que sus superiores los animan o por la existencia de requerimientos externos para cumplir con su trabajo. (González-Bravo & Valdivia-Peralta, 2015). Finalmente en el contexto de la educación superior, el TAM se ha aplicado con éxito para evaluar la adopción del e-learning, identificando que la autoeficacia o creencia en la propia capacidad técnica es un factor crítico para explicar la conducta del estudiante. Asimismo, las investigaciones indican que existen variables moderadoras, como el sexo, que influyen significativamente en cómo

el personal académico y el estudiantado perciben y adoptan las nuevas plataformas educativas (González-Bravo & Valdivia-Peralta, 2015).

Método Socrático

El método socrático es una forma de diálogo que utiliza la refutación para cuestionar lo que se cree saber. Su primer objetivo es "negativo". Es decir, busca destruir la falsa pretensión de conocimiento a través de preguntas, partiendo de una posición de ignorancia. Lo que implica que no se trata de una simple transmisión de información, sino de una transformación personal a través de la investigación compartida (Farieta-Barrera & Murillo, 2023)

En este trabajo en equipo, el guía no actúa como si tuviera todas las respuestas. Al contrario, reconoce que él también tiene dudas y se siente "atascado" frente al problema, para que junto al estudiante se encuentre el deseo de aprender a cuidar su integridad y actuar correctamente. Por lo tanto, el aprendizaje resultante no consiste en la acumulación de datos o conocimientos positivos, sino en el desarrollo de un metaconocimiento, es decir, una nueva relación con el propio saber y sus límites (Farieta-Barrera & Murillo, 2023). Este método es importante, ya que se espera que por medio del prompting, el sistema de tutoría inteligente lo use para enseñar a los estudiantes.

Supabase

Supabase es una plataforma conocida por ser backend as a service de código abierto, construida sobre el sistema de gestión de base de datos Postgres. Esta plataforma, esta pensada para desarrollar aplicaciones web y móviles con un conjunto integrado de herramientas listas para usar. La documentación oficial lo resume así: "Supabase is an open source Firebase alternative built on Postgres. It provides a complete backend platform for building web and mobile applications, with a suite of integrated tools that work together out of the box." En la práctica, eso significa que no es solo una base de datos: incluye autenticación, almacenamiento, funciones y APIs generadas automáticamente (S. Inc., 2026). Para el Proyecto, esa herramienta se tiene planteada como la base de datos.

Java

Java es un lenguaje de programación y un entorno diseñado para resolver desafíos en la programación moderna, habiéndose originado en un proyecto para crear software avanzado destinado a dispositivos electrónicos de consumo pequeños, portátiles y distribuidos («The Java TM Language : An Overview», 2009). Es definido como un lenguaje simple, orientado a objetos, distribuido, robusto y seguro, que además posee una arquitectura neutral y alta portabilidad. Su diseño como sistema de arquitectura neutral permite que las aplicaciones se ejecuten en cualquier lugar de una red, ya que el compilador genera instrucciones de bytecode que son fáciles de interpretar y traducir a código de máquina en cualquier plataforma («The Java TM Language : An Overview», 2009).

Java también enfatiza la fiabilidad mediante la comprobación temprana de errores, la eliminación de la aritmética de punteros que suele corromper datos y la inclusión de un sistema de recolección

automática de basura que simplifica la gestión de memoria para el programador. Finalmente, sus capacidades multihilo y dinámicas permiten que las aplicaciones se adapten a entornos en evolución y gestionen múltiples tareas simultáneas con una respuesta interactiva eficiente («The Java TM Language : An Overview», 2009). Este lenguaje es importante, dado que es el elegido para el desarrollo del backend de la plataforma.

Spring Boot

Spring Boot es una extensión del Spring Framework diseñada específicamente para simplificar el desarrollo de aplicaciones y permitir un tiempo de inicio mucho más rápido al escribir software (Deinum, 2018). Se define por adoptar una visión "opinada" (opinionated) de la plataforma Spring y de bibliotecas de terceros, lo que facilita que los desarrolladores comiencen sus proyectos con el mínimo esfuerzo y muy poca configuración manual. Su diseño permite crear aplicaciones independientes y de grado de producción que simplemente se pueden "ejecutar", eliminando gran parte de la complejidad tradicional del despliegue (Deinum, 2018).

Este marco de trabajo enfatiza la eficiencia mediante la autoconfiguración, la cual detecta automáticamente la presencia de diversas infraestructuras en el entorno como JDBC, JPA, RabbitMQ o Spring Security para configurarlas con valores predeterminados (Deinum, 2018). Asimismo, Spring Boot facilita enormemente el empaquetado y la gestión de dependencias al permitir la creación de archivos JAR ejecutables (conocidos como "fat JARs") que contienen todo el software necesario para funcionar de manera autónoma (Deinum, 2018). Finalmente, sus capacidades para el monitoreo, como Spring Boot Actuator, ofrecen puntos finales de salud y métricas listos para producción que permiten supervisar el estado del sistema de forma interactiva y eficiente (Deinum, 2018). Esta tecnología es fundamental, dado que es la elegida para optimizar y agilizar el desarrollo del backend de la plataforma.

Docker

Docker es una plataforma abierta diseñada para el desarrollo, envío y ejecución de aplicaciones. Su principal ventaja es que permite separar el software de la infraestructura subyacente, lo que acelera notablemente los tiempos de entrega. Al adoptar las metodologías de Docker para probar y desplegar código, las organizaciones pueden gestionar su infraestructura de la misma manera en que administran sus aplicaciones, reduciendo al mínimo la brecha temporal entre la escritura del código y su puesta en marcha en entornos de producción (D. Inc., 2026).

Para lograr esto, la plataforma empaqueta y ejecuta las aplicaciones dentro de entornos aislados y ligeros llamados contenedores. Esta separación y seguridad permiten ejecutar múltiples contenedores de forma simultánea en un mismo servidor. Dado que los contenedores son autónomos e incluyen absolutamente todo lo necesario para que la aplicación funcione, eliminan la dependencia de lo que esté instalado en el equipo de hosting. Además, garantizan que al compartir un contenedor con otros miembros del equipo, este funcionará exactamente de la misma manera para todos (D. Inc., 2026).

Finalmente, Docker ofrece las herramientas necesarias para gestionar por completo el ciclo de vida de estos contenedores. El proceso comienza utilizando los contenedores para desarrollar la aplicación y sus componentes de soporte, convirtiendo al contenedor en la unidad estándar de distribución y prueba. Una vez que el software está listo, se puede desplegar en el entorno de producción ya sea como un contenedor individual o como un servicio orquestado, manteniendo el mismo comportamiento sin importar si se implementa en un centro de datos local, en la nube o en un entorno híbrido (D. Inc., 2026).

Método CRAFT

El método CRAFT es un marco de trabajo centrado especificamente en docentes, que se diseña con el objetivo de crear prompts para la utilización de herramientas de Inteligencia Artificial como ChaGPT, Geminis o Claude. Su finalidad, es proveer a los profesores, especialmente a aquellos sin experiencia técnica en programación o en creación de prompts, una estrategia accesible y adaptada a su propio lenguaje profesional, para poder diseñar materiales de forma efectiva (Joshi et al., 2026). En pocas palabras, CRAFT es un modelo estructurado bajo un acrónimo en inglés de cinco elementos esenciales para diseñar instrucciones:

- **Context (Contexto):** Define el entorno de instrucción, especificando el nivel de grado, la materia y el tema (por ejemplo, una clase de ciencias sociales del colegio) (Joshi et al., 2026).
- **Role (Rol):** Asigna a la IA una identidad profesional o experiencia educativa específica (por ejemplo, un maestro experimentado en ciencias de primaria), lo que guía al modelo para producir contenido pedagógicamente adecuado (Joshi et al., 2026).
- **Audience (Audiencia):** Detalla las características de los estudiantes a los que va dirigida la lección, facilitando la diferenciación y la adaptación según sus necesidades individuales, dominio del idioma o conocimientos previos (Joshi et al., 2026).
- **Format (Formato):** Determina la estructura en la que se requiere que la IA entregue los materiales, como un plan de lección, una hoja de trabajo o una guía de discusión (Joshi et al., 2026).
- **Tone (Tono):** Asegura que la voz del contenido generado sea atractiva, adecuada para el desarrollo de los alumnos y congruente con la cultura del aula (por ejemplo, un tono curioso y alentador) (Joshi et al., 2026).

Método RCFOR (Role, Context, Format, Output, Reflection)

Es un marco de ingniería de prompts ágil y ligero a la vez que se diseña como alternativa flexible al modelo CO-STAR. Su propósito principal se centra en optimizar la interacción con modelos de IA de manera dinámica y eficiente. Dentro de sus pilares fundamentales se encuentran los siguientes:

- **Estructura Modular de 5 Componentes:** Divide la instrucción en Rol (identidad/persona), Contexto (antecedentes), Enfoque (atención prioritaria), Objetivo (meta exacta) y Respuesta (formato y síntesis final) (Pham & Ngoc, 2025).
- **Adaptabilidad Dinámica:** Permite ajustar el prompt en tiempo real según el contexto de la tarea, a diferencia de los moldes estáticos tradicionales (Pham & Ngoc, 2025).
- **Optimización e Iteración Automática:** Implementa un bucle de retroalimentación basado en principios de aprendizaje por refuerzo que reajusta automáticamente el Contexto, Enfoque y Objetivo ante respuestas de baja calidad, disminuyendo la intervención humana (Pham & Ngoc, 2025).
- **Buen Rendimiento:** En pruebas con GPT-4o-mini, obtuvo una precisión de 0.91 (casi igualando el 0.92 de CO-STAR), demostrando que es altamente efectivo pero con una estructura mucho más simple y escalable (Pham & Ngoc, 2025).

Método CO-STAR

Es un marco de ingeniería de prompts que se estrucutra y formaliza en la contrucción de un conjunto de intrucciones con el objetivo de lograr que sean más claras. Este es ampliamente conocido por su capacidad para guiar las respuestas de los modelos a través de plantillas sistemáticas predefinidas con anterioridad. Su estrucutra esta basada en un formato de promp compuesto por seis elemntos fiijos:

- **Context (Contexto)**: Proporcionar información de fondo ayuda al gestor de flujos a comprender el escenario específico y ofrecer respuestas pertinentes. (Craig et al., 2024)
- **Objective (Objetivo)**: Definir claramente la tarea orienta al gestor de flujos hacia el logro de ese objetivo específico. (Craig et al., 2024)
- **Style (Estilo)**: Especificar el estilo de redacción deseado, como imitar a una personalidad famosa o a un experto profesional, guía al gestor de flujos para que su respuesta se ajuste a sus necesidades. (Craig et al., 2024)
- **Tone (Tono)**: Establecer el tono garantiza que la respuesta transmita el sentimiento requerido, ya sea formal, humorístico o empático. (Craig et al., 2024)
- **Audience (Audiencia)**: Identificar la audiencia objetivo permite que el gestor de flujos adapte la respuesta para que sea apropiada y comprensible para grupos específicos, como expertos o principiantes. (Craig et al., 2024)
- **Response (Respuesta)**: Proporcionar el formato de respuesta, como una lista o JSON, garantiza que el gestor de flujos genere la respuesta en la estructura requerida para las tareas posteriores. (Craig et al., 2024)

2.2. Trabajos Relacionados

OATutor (Open Adaptive Tutor)

Es un sistema de tutoría adaptivo de codigo abierto basado en el algoritmo Bayesian Knowledge Tracing (BKT) que permite estimar la maestría del estudiante en componentes de conocimiento específico. Su propósito principal consta de democratizar las investigaciones en ciencias del aprendizaje, permitiendo que diversos investigadores y educadores experimenten con algoritmos de aprendizaje adapatativo sin la necesidad de tener conocimientos técnicos para crear plataformas educativas desde cero. Este también cuenta con un portal docente integrado en sistemas como Canvas que permite a los profesores monitorear hacer un seguimiento de la probabilidad de maestría por habilidad (Pardos et al., 2023).

Classroom Simulacra

Es un marco de trabajo que utiliza agentes generativos basados en LLMs para crear una especie de gemelo digital (estudiante) del aula, que permita imitar comportamientos de aprendizaje reales y predecir con precisión las posibles respuestas en pruebas posteriores de los estudiantes. Este marco de trabajo se apoya en una plataforma integrada denominada "CogEdu" que emplea técnicas de detección multimodales como el seguimiento ocular y el análisis de expresiones faciales con el objetivo de monitorear en tiempo real los estados cognitivos como la confusión y el nivel de atención. Además, este se centra en un innovador módulo de Reflexión Iterativa Transferible (TIR), que no solo permite que los profesores realicen seguimiento del proceso, sino que utilizan la simulación como un entorno de pruebas para optimizar estrategias de enseñanza (Xu et al., 2025). Esta implementación es un poco distinta a lo que se busca en el proyecto, ya que esta simula estudiantes virtuales con el objetivo de analizar y optimizar la forma en que aprenden para que investigadores y personas relacionadas al sector educativo puedan encontrar metodologías más precisas para enseñar y personalizar la educación.

CodeEdu

Es una plataforma de educación fundamentada principalmente en programación que utiliza un sistema de colaboración multiagente contruido sobre LLMs para ofrecer un aprendizaje proactivo y supervisado. Este se diferencia de los demás principalmente en que descompone tareas complejas y utiliza agentes especializados con el objetivo de guiar al estudiante en cuatro funciones principales: generación de material personalizado, resolución de dudas en tiempo real, tutoría de código paso a paso con depuración y generación de informes de aprendizaje (Zhao et al., 2025). Esta plataforma a diferencia de las demás no plantea un apartado para que el docente pueda hacer seguimiento a los estudiantes. Por último, es importante recalcar que el trabajo de esta plataforma todavía requiere de la revisión por pares.

2.3. Diferenciadores del Proyecto

- A diferencia de las soluciones analizadas en los trabajos relacionados, el proyecto se diferencia por lo siguiente:

  - **Configuración del Entorno Pedagógico mediante Prompting:** A diferencia de sistemas como OATutor, que dependen de algoritmos rígidos y estructuras de conocimiento predefinidas, este proyecto otorga al docente el control sobre el comportamiento del tutor. A través de estrategias de prompting, el profesor puede definir el tono, el nivel de complejidad y el dominio específico de conocimiento, asegurando que el sistema de tutoría inteligente se alinee con las necesidades del curso.
  - **Tutoría Socrática frente a Resolución Directa:** A diferencia de plataformas que actúan como depuradores automáticos o generadores de soluciones, este sistema implementa un modelo de diálogo socrático. El sistema de tutoría inteligente no entrega el código corregido, en su lugar, utiliza analogías y preguntas orientadoras.
  - **Dualidad Funcional (Estudiante-Docente):** Mientras que CodeEdu carece de herramientas de seguimiento para el profesor y Classroom Simulacra se limita a la simulación de alumnos virtuales, este proyecto ofrece un ecosistema completo. Por un lado, brinda un entorno de práctica dinámico para el estudiante y, por otro, un panel de monitoreo para el docente que permite identificar áreas críticas de mejora en el grupo, facilitando la toma de decisiones pedagógicas basadas en datos reales de interacción.

Capítulo 3

Desarrollo de la Plataforma

3.1. Revisión de Literatura para la Definición de Tipo de Feedback Requerido

Con el objetivo de poder identificar los requerimientos funcionales de la plataforma, se hace necesario identificar los tipos de feedback necesarios para que sean aplicados por el ITS. En la literatura académica, el feedback se define básicamente como la información proporcionada por un profesor a un estudiante sobre su desempeño actual con respecto a un objetivo estándar por alcanzar. A su vez, esta definición busca resolver directamente la brecha de retroalimentación (feedback gap) identificada en el planteamiento del problema, donde se señala que el estudiante no logra avanzar hacia las metas de aprendizaje por la falta de guías claras y recursos personalizados (Evans, 2013). En otras palabras, lo que se busca es utilizar un tipo feedback bien diseñado que intente cerrar esa brecha.

Por otra parte, cuando se habla del contexto de programación, este proceso es considerado bien exigente para los estudiantes desde un punto cognitivo, ya que los mismos deben construir, probar y revisar código de forma iterativa. Proceso, que en muchas ocasiones no es tan sencillo por lo que se requiere de una guía constante para desarrollar modelos mentales precisos sobre la ejecución de los programas (Almarales & Fernández, 2019; Evans, 2013). De acuerdo con lo investigado en revisiones de la literatura, la retroalimentación puede clasificarse según su nivel de complejidad y propósito instruccional:

- **Verificación (V):** Es la forma más simple, indicando únicamente si el resultado es correcto o incorrecto (Castellanos et al., 2026).
- **Respuesta Correcta (CR):** Provee al estudiante la solución esperada, lo cual es común en autocalificadores que muestran el código "referencia" después de que el alumno falla. El riesgo pedagógico de este tipo de feedback se fundamenta en que si se entrega demasiado pronto, puede fomentar que el estudiante simplemente copie la respuesta en lugar de comprender los conceptos subyacentes, entrenándolo para "responder la pregunta" en lugar de aprender a programar (Castellanos et al., 2026).
- **Retroalimentación Elaborada (E):** Este es el enfoque más recomendado pedagógicamente porque intenta explicar el "porqué" detrás de un resultado. No solo valida, sino que proporciona un proceso para que el estudiante reflexione (Castellanos et al., 2026). Dentro de esta categoría, los subtipos más importantes son:

- **Aislamiento de Atributos (AT):** Se enfoca en conceptos o atributos específicos del problema. Por ejemplo, en lugar de decir "tu código falló", el sistema señala, "tu error está en la definición de la jerarquía de clases", o "revisa cómo estás usando el encapsulamiento". Ayuda a reducir la carga cognitiva al dirigir la atención del alumno a un solo punto crítico (Castellanos et al., 2026).
- **Pistas o Señales (HC):** Funciona como un guía que no revela la solución, sino que orienta al alumno hacia ella. Utiliza mensajes como, "¿Has considerado qué sucede si la lista está vacía?", o "Revisa la condición de salida de tu ciclo while". Este método es clave para desarrollar habilidades de autorregulación y pensamiento crítico (Castellanos et al., 2026).

- **Retroalimentación Basada en Conceptos Erróneos (BM):** Este es quizás el tipo de feedback más sofisticado porque no trata el error como un simple fallo técnico, sino como la manifestación de una falla cognitiva o "idea errónea"(Castellanos et al., 2026; Gusukuma et al., 2018). Para lo anterior, se definen los siguientes elementos importantes:
  - **Diferencia entre error y concepto erróneo:** Un "error", es una configuración incorrecta de código, pero el "concepto erróneo", es la unidad de función cognitiva adquirida incorrectamente que lo causa (Gusukuma et al., 2018).
  - **Funcionamiento:** Si un estudiante confunde un "contador", con un "acumulador de suma", el feedback BM detecta ese patrón específico y lo anima a investigar la causa, por ejemplo "Parece que estás intentando sumar los valores de la lista en lugar de contar cuántos elementos hay ¿conoces la diferencia?"(Gusukuma et al., 2018).
  - **Impacto:** Se ha demostrado que este enfoque acelera significativamente la adquisición de conocimientos y habilidades prácticas, permitiendo que los estudiantes corrijan su modelo mental y no solo su sintaxis (Gusukuma et al., 2018). Permitiendo así que los estudiantes interioricen el conocimiento en lugar de solo "responder la pregunta"(Castellanos et al., 2026).

Siguiendo con la Retroalimentación Basada en Conceptos Erróneos, esta se fundamenta en la teoría del "componente de conocimiento". Bajo este modelo, un error de programación no se ve solo como una configuración incorrecta de código, sino como la manifestación de una falla cognitiva en el entendimiento del estudiante (Gusukuma et al., 2018). Al vincular los errores observados (como confundir un contador con un acumulador de suma) con vectores de conceptos erróneos específicos, es posible entregar mensajes que no solo corrijan el código, sino que modifiquen el pensamiento del alumno. La evidencia experimental indica que este tipo de feedback acelera significativamente la adquisición de conocimiento conceptual y habilidades prácticas, permitiendo que los estudiantes interioricen la lógica necesaria para resolver problemas sin depender exclusivamente de la asistencia docente (Gusukuma et al., 2018).

Por otra parte, con la evolución hacia el uso de Modelos de Lenguaje de Gran Tamaño (LLM) como GPT-4, el feedback ha pasado de ser estático y basado en reglas a ser dinámico, conversacional y altamente personalizado. Estos sistemas permiten la generación automática de comentarios en

lenguaje natural que analizan la intención, la lógica y la estructura del código en tiempo real, mejorando la legibilidad del mismo sin entregar la solución completa (Shekar et al., 2026). Se ha demostrado que este feedback es más efectivo cuando es específico, breve y fácil de entender, adaptándose al nivel actual de conocimientos del estudiante para evitar la sobrecarga cognitiva.

Finalmente la investigación sugiere que el feedback debe ser diseñado no solo para corregir errores sintácticos o fallos en casos de prueba, sino para fomentar procesos de aprendizaje de orden superior, como la autorregulación y el pensamiento algorítmico profundo. El impacto positivo de estas intervenciones se refleja no solo en el rendimiento académico y las notas, sino también en un aumento del compromiso y la persistencia de los estudiantes frente a tareas complejas (Castellanos et al., 2026).

3.2. Requerimientos de un Sistema de Tutoría Inteligente

Para desarrollar un Sistema de Tutoría Inteligente (ITS) para la enseñanza de programación, la literatura académica sugiere que el sistema debe estructurarse bajo un marco de cinco componentes fundamentales: el modelo del dominio (qué enseñar), el modelo del tutor (cómo enseñar), el modelo del estudiante (a quién se enseña), la interfaz y las ganancias de aprendizaje. Un buen ITS debe ser capaz de actuar como un tutor humano, poseyendo un conocimiento profundo de la materia y la capacidad de inferir el estado cognitivo del alumno para ofrecer una instrucción personalizada. En cuanto al núcleo pedagógico, un ITS efectivo suele basarse en el constructivismo, donde el sistema no solo entrega información, sino que actúa como un guía o "coach"que interviene para llenar vacíos de conocimiento o remediar errores detectados (Colby, 2017). Las tácticas específicas que debe incluir un buen sistema son:

- **Retroalimentación Adaptativa:** El sistema debe ofrecer feedback que no solo indique si una respuesta es correcta o no (Castellanos et al., 2026), sino que sea capaz de analizar los pasos intermedios del proceso de resolución o el resultado final (Colby, 2017).
- **Secuencias de Pistas (Hinting):** Es vital implementar secuencias de pistas que vayan de lo general a lo específico. Un buen ITS comienza con "pistas de puntero"que señalan un posible error sin dar la solución, seguidas de pistas didácticas, y terminan con una "pista de fondo"que proporciona la respuesta para evitar que el estudiante caiga en un ciclo de frustración (Colby, 2017).
- **Modelado del Estudiante:** El sistema debe contar con un modelo como el Rastreo de Conocimiento Bayesiano (BKT), para inferir continuamente qué habilidades ha dominado el alumno a partir de sus interacciones. Esto permite que el ITS sugiera la siguiente acción más apropiada cuando el estudiante está bloqueado (Kurup et al., 2016).
- **Andamiaje (Scaffolding):** El ITS debe proporcionar apoyo intensivo al inicio y retirarlo gradualmente a medida que el estudiante gana competencia, adaptando la dificultad de las tareas a la "zona de desarrollo próximo"del alumno (Colby, 2017).

Finalmente, la efectividad del feedback también depende críticamente del timing o momento de entrega. La retroalimentación inmediata o basada en la acción es la más prevalente (ya que ayuda a maximizar las oportunidades de aprendizaje y minimiza el tiempo dedicado a seguir caminos de solución incorrectos). Sin embargo, la investigación sugiere que el feedback debe ser diseñado no solo para corregir errores sintácticos o fallos en casos de prueba, sino para fomentar procesos de aprendizaje de orden superior, como la autorregulación y el pensamiento algorítmico profundo. El impacto positivo de estas intervenciones se refleja no solo en el rendimiento académico y las notas, sino también en un aumento del compromiso y la persistencia de los estudiantes frente a tareas complejas (Colby, 2017).

3.3. Definición de Requerimientos, Métricas, Módulos y Tecnologías de la Plataforma

A partir de la revisión de literatura realizada en las secciones anteriores, de una consulta con un asesor de innovación pedagógica y de preguntas hechas a los líderes de TI de la universidad, se definen los siguientes requerimientos funcionales, módulos y Tecnologías del prototipo de la plataforma.

3.3.1. Tecnologías de la Plataforma

Vale recalcar, que las tecnologías incialmente pensadas en el anteproyecto, se cambiaron dado que preguntando a los líderes de TI de la universidad, se utiliza un stack tecnológico diferente por lo que se busca estar lo más alineado posible. Estas son las siguientes:

Almacenamiento:

- Supabase (PostgreSQL)

Backend:

- Java (OpenJDK 21)
- Apache Maven
- Spring Boot

Frontend:

- Thymeleaf

Infraestructura:

- Docker

Despliegue/Hosting:

- Render/Servidor Privado

3.3.2. Módulos de la Plataforma

La siguiente sección comprende la descripción de las funcionalidades de los módulos planteados en apartados anteriores y otros ajustados o agrupados con lo investigado o preguntado a los stakeholders:

- **Interfaz de Interacción Socrática:** Este módulo comprende el chat con IA en donde el estudiante hará la interacción con el objetivo de aprender cualquier tema de programación organizado por el profesor. Además, la idea es que el estudiante pueda enviar feedback sobre la sección, por lo que se debe habilitar un apartado dentro del mismo módulo para que el estudiante pueda enviar sus opinions sobre la actividad realizada.
- **Panel de Métricas Docente:** En el módulo se presentan el conjunto de métricas y gráficas que le permiten al profesor conocer los avances que va teniendo el estudiante con el uso de la plataforma por temática. En este apartado vale aclarar que se utilizará IA para resumir la información presentada, dado que para muchos profesores puede ser difícil interpretar las métricas. Asimismo, el profesor podrá acceder a esta abriendo la sesión particular creada en donde podrá ver las métricas por esa sesión o en el apartado dentro del **Módulo de Administración de Profesor**, señalando un boton de métricas generales de todas las clases.
- **Panel de Configuración de Prompts con IA:** En consideración de lo conversado con el asesor pedagógico, para un profesor puede llegar a ser un poco difícil el hecho de tener que generar un buen prompt, por lo que se espera utilizar el método **método socrático** (Chang, 2023) en conjunto con el framework **CRAFT** (Joshi et al., 2026) para poder ayudarle a crear el prompt por temática. Lo que implica que este apartado es la unificación de los módulos "**Panel de Configuración de Prompts y Panel de Ayuda para Construcción de Prompts**" planteados en secciones anteriomente. Por úlitmo, es importante aclarar que este apartado es exclusivo del profesor y su acceso, se encuentra dentro del **Módulo de Administración de Profesor**.
- **Módulo de Inicio de Sesión:** Es el módulo correspondiente al inicio de sesión tanto para estudiantes como para profesores. En el caso de los profesores, estos pueden activar el multifactor para mayor seguridad. Para el tema del registro inicial, en ambos usuarios, esta etapa solo representa una variación del módulo actual.
- **Módulo de Administración de Profesor:** Este módulo es el principal del docente, en este el mismo puede crear sesiones de tutoría, accediendo al **Panel de Configuración de**

Prompts con IA. Puede acceder al **Panel de Métricas Docente** en donde podrá ver las métricas generales o particulares por sesión. Además, podrá ver toda su información personal, editarla y activar o desactivar el multifactor de autentificación y por úlitmo, podrá revisar el feedback que el estudiante esta enviando sobre la sesión.

- **Módulo de Interfaz de Estudiante:** Esta interfaz le permite al estudiante visualizar su información correspondiente al inicio de sesión y las clases a las cuales puede acceder.

3.3.3. Requisitos Funcionales

Cuadro 3.1: Requisitos Funcionales de la Plataforma

ID	Módulo	Descripción	Fuente
RF-01	Módulo de Interfaz de Estudiante, Módulo de Inició de Sesión, Módulo de Administración de Profesor.	El docente crea una sesión de tutoría, que automáticamente genera un enlace único de acceso a la sesión. Al abrirlo, el estudiante completa un registro mínimo con su nombre y correo institucional, quedando vinculado automáticamente a la sesión sin pasos adicionales.	Stakeholder
RF-02	Módulo de Interfaz de Estudiante, Interfaz de Interacción Socrática.	El sistema debe permitir que el estudiante interactúe con el ITS mediante un chat conversacional que aplique el método socrático, entregando retroalimentación elaborada a través de pistas (HC). El ITS no debe revelar la solución directa, sino guiar al estudiante para que la descubra por sí mismo a través del diálogo.	(Castellanos et al., 2026)
RF-03	Interfaz de Interacción Socrática.	El sistema debe entregar retroalimentación inmediata tras cada interacción del estudiante, apoyando los ciclos rápidos de ensayo y error esenciales para el aprendizaje de programación.	(Castellanos et al., 2026)
RF-04	Interfaz de Interacción Socrática.	El sistema debe almacenar el historial de conversaciones de cada estudiante para que el ITS pueda retomar sesiones previas, reconocer patrones de error recurrentes y adaptar su discurso de forma evolutiva.	(Colby, 2017; Kurup et al., 2016)

Continúa en la siguiente página

Cuadro 3.1 – Continuación

ID	Módulo	Descripción	Fuente
RF-05	Panel de Configuración de Prompts con IA.	El docente debe poder configurar el comportamiento socrático del ITS mediante un paso a paso estructurado ayudado con IA, que permita definir el tema, el nivel de complejidad y el tono del tutor, sin necesidad de conocimientos técnicos previos en ingeniería de prompts.	Stakeholder
RF-06	Panel de Métricas Docente.	La plataforma debe mostrar al docente los conceptos con mayor frecuencia de error a nivel individual y grupal, permitiéndole identificar las falencias más críticas del grupo.	(Colby, 2017)
RF-07	Panel de Métricas Docente.	El sistema debe mostrar la evolución del desempeño de cada estudiante entre sesiones, indicando si está mejorando o persistiendo en los mismos errores.	(Colby, 2017; Kurup et al., 2016)
RF-8	Panel de Métricas Docente.	La plataforma debe generar un resumen automático en lenguaje natural que interprete las métricas del grupo y sugiera al docente acciones pedagógicas concretas, sin necesidad de que el profesor interprete los datos por su cuenta.	Stakeholder
RF-9	Panel de Métricas Docente.	El sistema debe registrar el número de sesiones, la duración promedio y la frecuencia de uso por estudiante, para que el docente pueda identificar estudiantes en riesgo de abandono o desenganche.	Stakeholder
RF-10	Interfaz de Interacción Socrática.	El ITS debe rechazar internamente cualquier respuesta que contenga la solución directa al problema del estudiante, reformulándola como una pregunta orientadora antes de entregarla.	(Shekar et al., 2026)

3.3.4. Requisitos No Funcionales

Los siguientes dos requisitos no funcionales (RNF), están enfocados principalmente en preservar la confidencialidad de la información entre el estduiante y el profesor en la plataforma.

Cuadro 3.2: Requisitos No Funcionales de la Plataforma

ID	Módulo	Descripción	Fuente
RNF-01	Módulo de Interfaz de Estudiante, Interfaz de Interacción Socrática.	El sistema deberá restringir el acceso a las conversaciones de un estudiante exclusivamente a dicho estudiante y al docente responsable de la sesión correspondiente, de modo que solo el estudiante y el profesor puedan tener acceso al chat de tutoría respectivo.	(International Organization for Standardization, 2022)
RNF-02	Módulo de Inicio de Sesión, Módulo de Administración de Clase.	El sistema deberá almacenar únicamente los datos personales mínimos necesarios para la operación, tales como nombre y correo institucional, los cuales el estudiante registra una única vez al ingresar a una nueva sesión de tutoría.	(International Organization for Standardization, 2022)

3.3.5. Métricas de Seguimiento

Las siguientes métricas serán recolectadas por la plataforma para alimentar el Panel de Métricas Docente. En esse sentido cada métrica está acompañada de su traducción pedagógica, es decir, lo que significa en términos prácticos para el profesor:

Cuadro 3.3: Métricas de Seguimiento de la Plataforma

ID	**Métrica**	**Descripción técnica**	**Traducción pedagógica**	**Fuente**
M-01	Frecuencia de error por concepto	Número de veces que un concepto específico genera un error en el grupo durante una sesión.	"Este es el tema que más le está costando al grupo. Considera dedicarle más tiempo en clase."	(Colby, 2017)
M-02	Intentos por concepto	Número de turnos de conversación que necesita el estudiante antes de llegar a la comprensión del concepto.	"Este estudiante necesita muchos intentos para entender el concepto. Puede requerir atención personalizada."	(Colby, 2017; Kurup et al., 2016)
M-03	Evolución entre sesiones	Comparación del desempeño del estudiante en el mismo concepto entre sesiones distintas.	"Este estudiante mejoró / sigue fallando en el mismo punto que la semana pasada."	(Kurup et al., 2016)
M-04	Tasa de solicitudes de respuesta directa	Número de veces que el estudiante pide explícitamente la solución durante la sesión.	"Este estudiante muestra señales de frustración. Puede necesitar una estrategia diferente."	(Castellanos et al., 2026)
M-05	Frecuencia y duración de sesiones	Número de sesiones completadas y tiempo promedio por sesión por estudiante.	"Este estudiante no ha usado la plataforma en varios días. Puede estar en riesgo de abandono."	(Castellanos et al., 2026)

3.4. Decisiones de Diseño de la Arquitectura de Prompts y Persistencia

3.4.1. Arquitectura de Prompts y Persistencia de Datos

Con el objetivo de definir un flujo de datos consistente con los objetivos en la plataforma, es necesario evaluar distintos enfoques para estructurar el prompt que recibe el modelo de lenguaje en cada interacción con el estudiante. A continuación se documentan las decisiones tomadas respecto al framework de **prompting** que se espera utilizar y la justificación detrás de su elección.

Arquitectura en Capas frente a un Prompt Monolítico

Inicialmente se consideró construir el prompt del sistema como un único bloque de texto que combinara las reglas socráticas de comportamiento del tutor, la configuración particular del docente y el historial de progreso del estudiante. Sin embargo, este enfoque presenta dos limitaciones importantes. La primera se da en la dificultad del mantenimiento del sistema, ya que cualquier ajuste a las reglas pedagógicas obligaría a revisar un solo bloque de texto extenso y no permite que el docente configure la sesión sin riesgo de alterar accidentalmente las restricciones fijas del método socrático.

Por esta razón, se decidió estructurar el prompt en tres capas independientes, cada una con un origen, un propósito y una frecuencia de cambio distintos. La primera es una capa fija que define el comportamiento socrático del sistema, la segunda es una capa configurable que el docente ajusta por sesión y la última, aunque no es un prompt en sí, es una capa dinámica que recoge el progreso individual del estudiante. Esta separación permite que cada capa se mantenga, audite y modifique de manera independiente, sin comprometer las demás. Es importante aclarar que, cuando se menciona cada elemento como una capa, se habla de textos en formato Markdown que definen el comportamiento y parte del aprendizaje del estudiante. Lo que cambia en realidad es el lugar donde se ubica cada capa y si el docente la configura o no.

Capa Fija: Método Socrático

La primera capa contiene las reglas de comportamiento que el tutor debe seguir en todas las sesiones, sin importar el tema o el estudiante. Esta capa no es visible ni modificable por el docente, ya que su función es garantizar que el sistema mantenga siempre el enfoque pedagógico central del proyecto, es decir, guiar al estudiante mediante preguntas y analogías en lugar de entregar soluciones directas, tal como lo plantea el método socrático (Chang, 2023).

Se decidió mantener esta capa separada de la configuración del docente precisamente para evitar que un ajuste mal formulado en el panel de configuración pudiera debilitar o eliminar las restricciones pedagógicas fundamentales del sistema. De esta manera, el docente tiene control sobre el contenido y el enfoque temático de la sesión, pero no sobre los principios pedagógicos que sostienen el comportamiento socrático del tutor.

Capa Configurable: Adopción del Marco CRAFT

Para la capa que el docente configura por sesión, se evaluaron tres frameworks de **prompting** orientados al diseño de instrucciones estructuradas: **CRAFT** (Joshi et al., 2026), **CO-STAR** (Craig et al., 2024; Pham & Ngoc, 2025) y **RCFOR** (Pham & Ngoc, 2025). Tras comparar la naturaleza y el propósito de cada uno, se optó por adoptar **CRAFT** (Context, Role, Audience, Format, Tone) como base de esta capa, dado que es el único de los tres frameworks diseñado y validado específicamente para docentes sin experiencia técnica en ingeniería de prompts. Su validación empírica con 94 docentes de educación básica que generaron 688 prompts reales para la creación de planes de clase y material instruccional lo posiciona como la opción con mayor evidencia de usabilidad para el perfil de usuario que compone el módulo de configuración de sesión de la plataforma (Joshi et al., 2026).

Esta elección no descarta los aportes de los otros dos frameworks evaluados, sino que reconoce su alcance distinto. **CO-STAR**, con sus seis componentes (Context, Objective, Style, Tone, Audience, Response), presenta el desempeño más alto en tareas de generación de contenido y clasificación bajo evaluación experimental directa frente a RCFOR, con métricas superiores en precisión, relevancia y coherencia (Pham & Ngoc, 2025). Sin embargo, ni CO-STAR (Craig et al., 2024) ni RCFOR (Pham & Ngoc, 2025) cuentan con validación en un contexto pedagógico ni con usuarios docentes reales. Ambos fueron evaluados sobre tareas genéricas de procesamiento de lenguaje natural, lo cual limita su pertinencia directa para el módulo de configuración dirigido al profesor. **RCFOR**, por su parte, destaca por la separación explícita de su componente Focus respecto al componente Objective, una característica estructural que resulta valiosa para el mecanismo de validación interna descrito en la capa fija, en el que el sistema debe verificar de forma aislada si la restricción de no revelar soluciones directas (RF-10) se cumplió, sin necesidad de reconstruir el prompt completo (Pham & Ngoc, 2025). No obstante, dado que dicha restricción ya se encuentra respaldada de forma centralizada en la capa fija del sistema, esta separación de RCFOR no resulta indispensable para justificar su adopción como framework principal de la capa configurable.

En consecuencia, la plataforma implementa CRAFT (Joshi et al., 2026) como la estructura que el docente completa a través del formulario de configuración de sesión, mientras que la responsabilidad de restringir la generación de soluciones directas recae en la capa fija del sistema, descrita en la sección anterior y no en un componente específico de la capa configurable. Para este punto, se anexa una tarea específica para el profesor, el cual tiene el trabajo de definir un conjunto de preguntas que sirvan de guía para el tutor y que al mismo tiempo, le permitan al estudiante aprender lo necesario por sesión. Aquí cabe aclarar que el tutor seguirá cumpliendo la labor de brindar feedback al estudiante, así como también responder las diferentes preguntas que le puedan surgir al mismo en el progreso, pero este nueva actividad se anexa, dado que sin ello, como al tutor no se le da un contexto del tema, puede tender a divagar más en la temática y no concentrarse en lo que el docente en su experiencia considera que el estudiante pueda necesitar entender.

Esquema de Persistencia del Progreso, Métricas y Preguntas Orientadoras

Antes de definir el esquema final de persistencia, se evaluaron tres alternativas para almacenar el progreso del estudiante y las métricas de seguimiento del sistema. La primera alternativa consistía en utilizar tablas estructuradas en Supabase, con columnas específicas para cada dato del análisis, como el concepto trabajado, si el estudiante entendió al final, el nivel de comprensión, el número de intentos y los errores recurrentes. Esta alternativa facilita el filtrado y la consulta directa mediante SQL en el panel docente, pero exige construir manualmente el texto que se inyecta en el prompt a partir de los valores de cada columna y obliga a modificar la estructura de la tabla cada vez que cambia el tipo de análisis requerido.

La segunda alternativa consistía en almacenar el análisis generado por la API directamente como una cadena de texto en formato JSON, dentro de una única columna de tipo texto. Esta alternativa permite que el modelo devuelva la información ya estructurada sin que el backend deba construirla manualmente y ofrece mayor flexibilidad ante cambios en el análisis, dado que no requiere alterar el esquema de la base de datos. Sin embargo, exige un paso adicional de deserialización mediante una librería como Jackson antes de poder utilizar la información, tanto para mostrarla en el panel docente como para inyectarla en el prompt de la siguiente sesión y expone al sistema al riesgo de recibir una respuesta con un formato JSON mal construido por parte del modelo.

La tercera alternativa consistía en almacenar el análisis generado por la API como texto plano con sintaxis Markdown, en una columna de la tabla de estudiantes. Esta alternativa permite que el contenido se inyecte directamente en el prompt de la siguiente sesión sin ningún tipo de transformación adicional y resulta igualmente legible tanto para el modelo como para una persona que revise el dato manualmente. Su principal limitación radica en que resulta más difícil de filtrar o consultar de forma selectiva desde el panel docente, en comparación con columnas estructuradas, y su tamaño puede crecer de manera considerable a medida que se acumulan sesiones.

Con base en esta comparación, se optó por adoptar una combinación entre la primera y la tercera alternativa, descartando el uso de JSON como formato intermedio. Las métricas de seguimiento descritas en la Sección 3.3.5 (Cuadro 3.3) se almacenan en columnas estructuradas de tipo numérico o booleano dentro de Supabase, vinculadas al estudiante y a la sesión correspondiente y son generadas y actualizadas por Spring Boot en tiempo real mediante contadores que se incrementan con cada interacción del estudiante, tales como los intentos por concepto y las solicitudes de respuesta directa. Este formato permite consultas directas mediante SQL y facilita la construcción de gráficas y filtros en el panel de métricas docente, sin requerir procesamiento adicional sobre el dato almacenado.

El progreso cualitativo del estudiante, en cambio, se almacena como texto con sintaxis Markdown en la columna `progreso_md` de la tabla `estudiante`. Este texto es generado por la API al finalizar cada sesión, a partir del análisis completo de la conversación y contiene secciones predefinidas, como conceptos dominados, conceptos en progreso, errores recurrentes y recomendación para la siguiente sesión. Se optó por este formato en lugar de la estructura JSON evaluada inicialmente, debido a que el texto en Markdown puede inyectarse directamente en el prompt de la siguiente sesión sin necesidad de deserialización ni transformación adicional. La necesidad de filtrado y consulta que originalmente motivaba el uso de JSON para el panel docente quedó cubierta por las

columnas numéricas estructuradas de las métricas de seguimiento, por lo que mantener una tercera representación del dato en formato JSON puede introducir una capa adicional de complejidad sin un beneficio claro sobre el esquema finalmente adoptado.

Es importante precisar que, aunque este contenido se describe como un archivo en formato Markdown, no corresponde a un archivo físico almacenado en el sistema de archivos del servidor, sino a una cadena de texto almacenada como el valor de una columna de tipo texto en una fila de la base de datos relacional. Esta distinción resulta relevante desde el punto de vista de la arquitectura de despliegue, dado que el contenedor de Docker en el que se ejecuta la aplicación es efímero y cualquier archivo escrito directamente en el sistema de archivos se perdería al reiniciarse o reconstruirse el contenedor, mientras que la información almacenada en Supabase persiste de manera independiente al ciclo de vida del contenedor de la aplicación.

Capa Dinámica

La capa dinámica del prompt incorpora el progreso del estudiante como una tercera fuente de contexto, complementaria al historial de la conversación activa. Su construcción depende de si existe o no una participación previa del estudiante sobre el mismo tema.

Cuando el estudiante inicia una nueva sesión, Spring Boot consulta si existe una participación anterior de esa misma persona en una sesión con el mismo tema. Si dicha participación existe, se recupera el campo `progreso_md` generado al cierre de esa sesión previa y se incorpora directamente a la capa dinámica del prompt, sin ninguna transformación adicional. Si el estudiante no registra participaciones previas sobre el tema, no existe aún un `progreso_md` que heredar, en ese caso, el progreso del estudiante durante la sesión activa se construye únicamente a partir de los mensajes que este intercambia con el sistema, los cuales sirven como insumo para el análisis que se generará al finalizar la sesión.

En ambos casos, el historial de mensajes de la sesión activa cumple una doble función, por un lado, se inyecta turno a turno en la capa dinámica para que el modelo de seguimiento a la evolución del estudiante dentro de la conversación en curso, por otro, constituye la base sobre la cual la API genera, al cierre de la sesión, el resumen estructurado que se persistirá como el nuevo `progreso_md` de esa participación. En cuanto a la estructura del progreso al cierre de sesión, con el fin de que el contenido de `progreso_md` sea consistente entre sesiones y pueda inyectarse de forma predecible en la capa dinámica de sesiones futuras, el análisis generado por la API al finalizar cada sesión debe seguir estrictamente la siguiente estructura en formato Markdown:

```
## Conceptos dominados
- [Concepto 1]
- [Concepto 2]

## Conceptos en progreso
- [Concepto 1]
- [Concepto 2]

## Errores recurrentes
```

```
- [Descripción del error 1]
- [Descripción del error 2]

## Recomendación para la siguiente sesión
- [Recomendación puntual y accionable]
```

Esta estructura fija garantiza que, sin importar cuántas veces se regenere el progreso a lo largo de distintas sesiones, el contenido conserve siempre las mismas cuatro secciones, permitiendo que tanto el modelo, al recibirlo como contexto en una sesión futura, como el docente, al visualizarlo en el panel de monitoreo, puedan interpretarlo de forma consistente sin necesidad de un análisis adicional sobre su formato.

3.4.2. Definicion de API de IA a Utilizar

En este apartado se realiza el análisis de las APIs de IA disponibles para utilizar. Para ello se utiliza la página Artificial Analysis (Artificial Analysis, 2026), una plataforma independiente de evaluación comparativa de modelos de IA que no pertenece a ningún proveedor particular, por lo que goza de mayor objetividad. Asimismo, esta página cuenta con un índice compuesto denominado Artificial **Analysis Intelligence Index**, que integra un conjunto de evaluaciones de rendimiento, entre las que se encuentran GDPval-AA v2, Terminal-Bench v2.1, SciCode, Humanity's Last Exam, GPQA Diamond, CritPt, AA-Omniscience, AA-LCR y LiveCodeBench.

Por otra parte, dado que las APIs en su mayoría requieren de pago por su uso y el proyecto se costea con recursos propios, la comparación se hace escogiendo modelos representativos de cada uno de los principales proveedores comerciales, con la consideración de siemppre eligir la mejor relación entre capacidad y costo para el desarrollo que se esta haciendo. En ese sentido se eligen los siguientes modelos para la evaluación, Gemini 3 Flash (Google), GPT-5 mini (high) (OpenAI), Grok 4 Fast (xAI) y Claude 4.5 Haiku (Anthropic).

Como se puede ver en la figura (3.1) del **Analysis Intelligence Index**, Gemini 3 Flash de Google gana con una puntuación de 39 puntos, seguido de Claude Haiku con 30 y Grok y ChatGpt con 28 y 26 puntos, una puntuación casi similar, que pone dentro de los modelos de gama media-baja a Gemini 3 Flash unos puntos por arriba y que al mismo tiempo lo adecua dentro de las necesidades del proyecto dado que se valida su buen rendimiento general en **benchmarks** de código y exámenes de conocimiento humano.

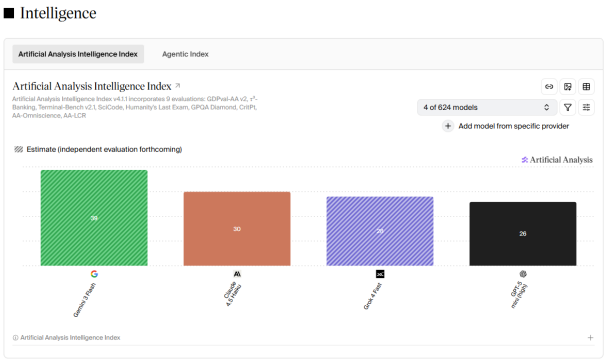


Figura 3.1: Artificial Analysis Intelligence Index

Por otra parte, cuando nos basamos particularmente en el **LiveCodeBench**, que es un sistema de evaluación comparativa de codificación libre de contaminación que recopila continuamente nuevos problemas de programación competitiva de LeetCode, AtCoder y CodeForces, con el objetivo de evaluar la generación de código, la autorreparación y la ejecución, que al final es el objetivo de la plataforma, enseñar temas de codificación, podemos ver la figura (3.2) en la que Geminis casi por un poco menos de 7 % supera a los demás modelos.

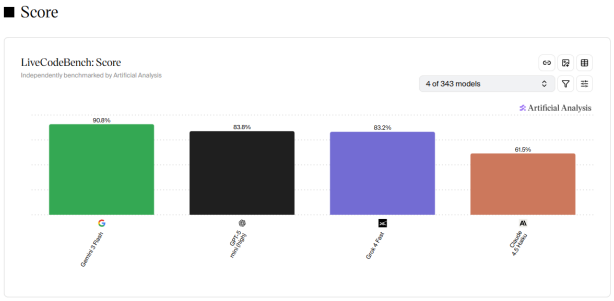


Figura 3.2: LiveCodeBench: Score

Siguiendo con los temas de costos, si evaluamos la figura (3.3), se puede notar, en cuanto al costo por millón de tokens que Grok 4 Fast presenta el precio más bajo del grupo con 0,2$ en tokens de entrada y 0,5$ en tokens de salida, seguido de GPT-5 mini (high) con 0,25$ y 2$ respectivamente, y Gemini 3 Flash, con 0,5$ y 3$. Por otra parte, Claude 4.5 Haiku resulta el más costoso de los cuatro, con un precio de salida de 5$ por millón de tokens, diez veces superior al de Grok 4 Fast. Lo que implica que aún con un costo intermedio, Geminis 3 Flash representa el mejor costo beneficio para el proyecto.

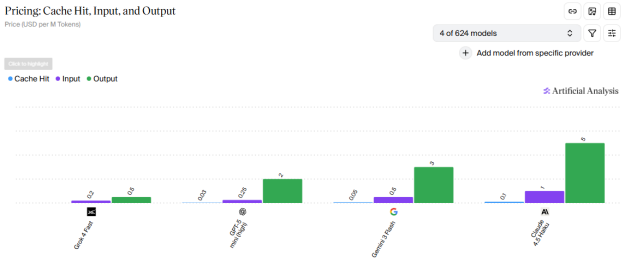


Figura 3.3: Pricing: Cache Hit, Input, and Output

Por último, si bien no se encontraron métricas ligadas directamente a contextos educativos, algunos de los benchmarks que componen el Intelligence Index sí evalúan capacidades afines, como GPQA Diamond, que mide la resolución de problemas de nivel de posgrado en biología, física y química. Por otro lado, también esta el Humanity's Last Exam, que evalúa conocimiento y razonamiento experto a través de un amplio rango de disciplinas académicas.

Cabe recalcar que modelos de gama alta como Claude Opus 5 o Claude Fable 5 podrían superar a los otros modelos evaluados de las en los campos requeridos. Sin embargo, su costo considerablemente mayor motiva que la comparación se centre en la gama económica de cada proveedor, dado que las pruebas del proyecto correrán con recursos propios. Asimismo, con base en los resultados obtenidos, se selecciona Gemini 3 Flash como la API a utilizar en el proyecto, al ofrecer el mejor desempeño tanto en LiveCodeBench como en el Intelligence Index dentro de la gama económica evaluada, manteniendo además un costo intermedio frente a las demás alternativas consideradas. Por úlitmo, toda la información de métricas presentada en este apartado puede consultarse en (Artificial Analysis, 2026).

3.4.3. Flujos de Datos y Persistencia

Con el objetivo de establecer cómo se mueve la información entre los componentes del sistema antes de iniciar el desarrollo, se definen a continuación los flujos de datos principales de la plataforma y el esquema de persistencia en Supabase.

Flujo 1: Configuración de sesión bajo el marco CRAFT

1. El docente inicia sesión en la plataforma, con la opción de autenticación multifactor activada o desactivada, y accede al Módulo de Administración de Profesor.

2. El docente configura el prompt del ITS mediante el Panel de Configuración de Prompts con IA, diseñado bajo la estructura CRAFT, definiendo sus cinco componentes. Contexto, correspondiente al tema y la materia a enseñar. Rol, que define qué tipo de tutor debe ser el sistema, por ejemplo un tutor socrático paciente. Audiencia, que recoge las características del estudiante al que va dirigida la sesión. Formato, que establece cómo debe ser la salida de la conversación y por úmitmo el tono, que define la actitud que debe mantener el chat. Estos

componentes se configuran de manera independiente al conjunto fijo de reglas del método socrático, el cual reside en un archivo de instrucciones del sistema y no es modificable por el docente.

3. Adicionalmente, el docente define un conjunto de preguntas guía para la sesión, las cuales orientan al tutor sobre los puntos concretos que el estudiante debe comprender, sin restringir su capacidad de responder dudas adicionales que este pueda plantear durante la conversación. Estas preguntas, al igual que los cinco componentes del CRAFT, forman parte de la capa configurable del prompt.

4. Spring Boot almacena los cinco componentes del CRAFT y las preguntas guía de forma modular en columnas independientes de Supabase, y genera un enlace único de acceso asociado a la sesión.

5. El docente comparte el enlace con sus estudiantes.

6. El estudiante abre el enlace en su navegador y Spring Boot valida el identificador contra Supabase. Si el enlace es válido y la sesión está activa, se presenta al estudiante un formulario de registro mínimo con su nombre y correo institucional. Si el enlace es inválido, ha expirado o la sesión fue cerrada por el docente, se muestra un mensaje de error y se deniega el acceso.

7. Al completar el registro, el estudiante queda vinculado automáticamente a la sesión y se habilita su acceso al chat socrático, dentro de la Interfaz de Interacción Socrática.

Flujo 2: Interacción del estudiante

1. El estudiante envía un mensaje a través de la interfaz de chat desarrollada con Thymeleaf.

2. Spring Boot consulta en Supabase el historial de la conversación de la sesión activa, y determina el progreso a inyectar en la capa dinámica del prompt según la lógica descrita anteriormente. Si existe una participación previa del estudiante sobre el mismo tema, se recupera el `progreso_md` generado al cierre de dicha sesión y se incorpora directamente, sin ninguna transformación adicional. Si no existe una participación previa sobre el tema, no hay `progreso_md` que heredar, y el progreso del estudiante se construye únicamente a partir de los mensajes intercambiados durante la sesión activa.

3. Spring Boot construye el prompt combinando tres capas. La Capa 1, con las reglas fijas del método socrático definidas en el archivo del sistema. La Capa 2, con los cinco componentes del CRAFT y las preguntas guía configuradas por el docente para la sesión y por úlitmo, la Capa 3, con el contexto dinámico compuesto por el progreso heredado o en construcción del estudiante, el historial de la sesión activa y el mensaje nuevo.

4. Spring Boot envía el prompt completo a la API, que responde aplicando el método socrático mediante preguntas y analogías, siguiendo las reglas fijas de la Capa 1, el estilo definido por el docente en el CRAFT y orientándose hacia las preguntas guía definidas para la sesión.

5. Spring Boot almacena en Supabase el mensaje del estudiante y la respuesta entregada, además de registrar en columnas de formato numérico las métricas de seguimiento asociadas a la interacción, como los intentos por concepto y las solicitudes de respuesta directa.

6. La respuesta se muestra al estudiante en la interfaz.

Flujo 3: Cierre de sesión, generación de progreso y optimización pedagógica

1. Al finalizar la sesión, Spring Boot solicita a la API un análisis de la conversación completa, tomando como insumo el historial de mensajes intercambiados durante la sesión activa. Este análisis se recibe como texto con formato Markdown, siguiendo estrictamente la estructura fija de cuatro secciones descrita en la Capa Dinámica y se almacena en el campo `progreso_md` de la participación correspondiente en Supabase.

2. Las métricas numéricas de cada interacción se consolidan en Supabase dentro de columnas estructuradas, vinculadas al estudiante y a la sesión correspondiente.

3. Cuando el docente accede al Panel de Métricas Docente, ya sea desde una sesión particular o desde la vista general dentro del Módulo de Administración de Profesor, el sistema muestra las métricas agregadas del grupo, lo que permite identificar temas en donde el estudiante pueda estar fallando.

4. El Panel de Métricas Docente muestra las métricas agregadas del grupo, el resumen generado por la inteligencia artificial y las sugerencias pedagógicas derivadas de este, sin afectar en ningún caso las reglas fijas del método socrático ni las preguntas guía definidas por el docente, las cuales permanecen constantes durante toda la sesión.

Qué se envía a la API en cada llamada

Con el fin de precisar el contenido exacto de cada llamada a la API descrita en los flujos anteriores, se distinguen dos momentos.

En cada interacción del estudiante, correspondiente al Flujo 2, se envía el prompt completo, compuesto por la Capa 1, con las reglas fijas del método socrático. La Capa 2, con los cinco componentes del CRAFT y las preguntas guía de la sesión activa y la Capa 3, con el `progreso_md` heredado de una sesión previa sobre el mismo tema, si existe, el historial completo de mensajes de la sesión activa y el mensaje nuevo del estudiante.

Al cierre de la sesión, correspondiente al Flujo 3, se envía únicamente el historial completo de mensajes de la sesión que finaliza, junto con la instrucción de generar el análisis estructurado en la plantilla Markdown de cuatro secciones definida en la Capa Dinámica. Esta llamada no incluye la Capa 1 ni la Capa 2, dado que su propósito no es sostener el diálogo socrático, sino sintetizar la conversación ya concluida.

3.5. Modelo C4

3.5.1. Diseño de Diagrama de Contexto

La figura (3.4) presenta el diagrama de contexto del sistema, que busca mostrar una visión general de la solución con su alcance delimitado. Para este nivel de diagramación se puede notar que los agentes externos son el profesor, que crea clases, adiciona estudiantes a los mismos y revisa su resultados en un panel de métricas. Están los estudiantes, que interactúan con la plataforma y reciben retroalimentación socrática. Por último, la plataforma también se conecta a sistemas externos, como lo son la API de IA que es a la cual se encarga de procesar la respuesta de los estudiantes para poder generar retroalimentación y también se encuentra la base de datos de SupaBase que se encarga de guardar la información de los usuarios.

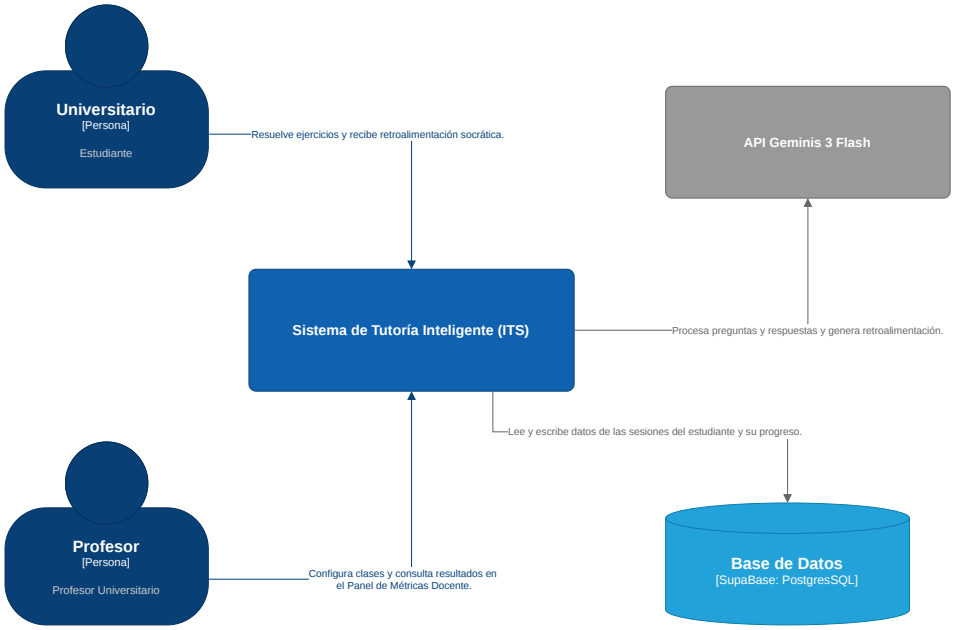


Figura 3.4: Diagrama de Contexto

3.5.2. Diagrama de Contenedores

La figura (3.5) presenta el diagrama de contenedores del proyecto. Por una parte están los agentes externos que son los profesores y estudiantes de los que se hablo antes. También está los contenedores principales como el de **Aplicación Web**, este se desarrolla en **Spring Boot + Thymeleaf** y se encarga de proporcionar la interfaz con todas las funcionalidades de retroalimentación, monitoreo de avance de los estudiantes y demás funcionalidades antes comentadas. Además, se encuentra el contenedor de **Módulo de Orquestación de IA** que también se construye en **Spring Boot** y se encarga de la interacción con la IA que para el proyecto es **Geminis 3 Flash** y que a su vez se conecta con el tercer contenedor que es el de la base de datos que sería de **PostgreSQL** que se utiliza para almacenar la información de los usuarios incluyendo el avance que van teniendo los estudiantes.

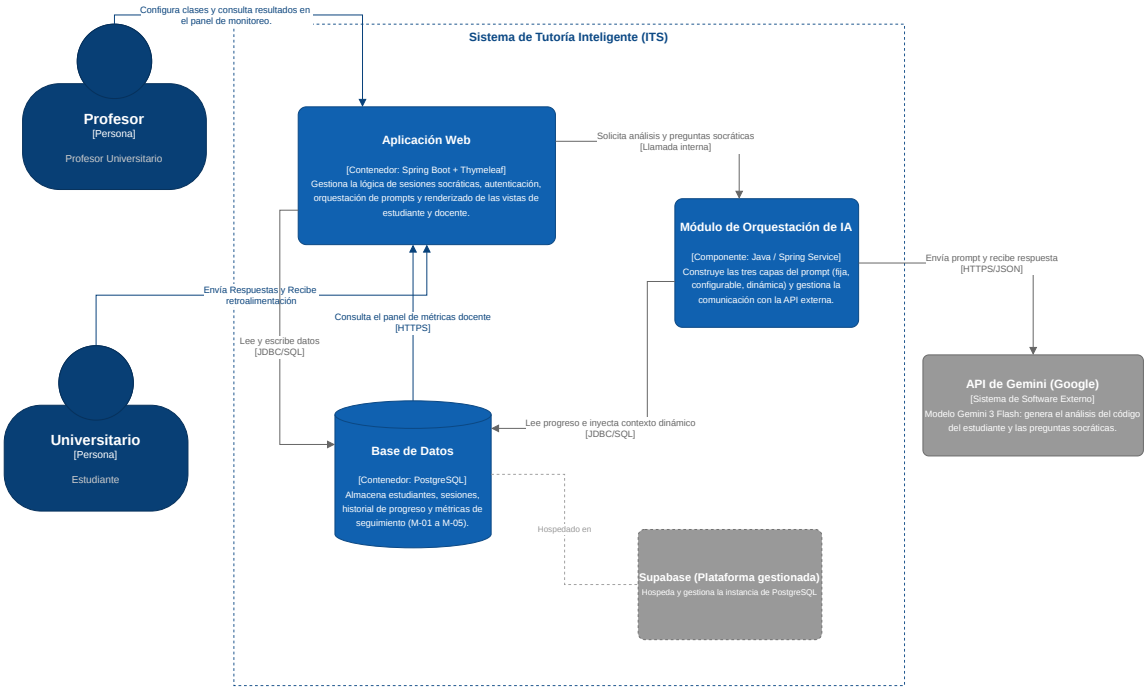


Figura 3.5: Diagrama de Contenedores

3.5.3. Diagrama de Componentes

Como se observa en la figura (3.6), el Módulo de Orquestación de IA se organiza en dos flujos principales, la construcción del prompt y la generación validada de retroalimentación. La construcción es coordinada por el componente Controlador de Sesión Socrática, el cual recibe la solicitud desde la Aplicación Web y delega el ensamblaje al Constructor de Prompt. Este último combina tres fuentes: la Capa Fija, que aporta las reglas del método socrático, la Capa Configurable, que aplica los cinco componentes del framework CRAFT definidos por el docente más las preguntas para la sesión y la Capa Dinámica, que obtiene del Repositorio de Progreso el historial de mensajes y el progreso en Markdown del estudiante. Una vez ensamblado, el prompt se envía al Cliente de API de IA, encargado de comunicarse con el servicio externo de Gemini.

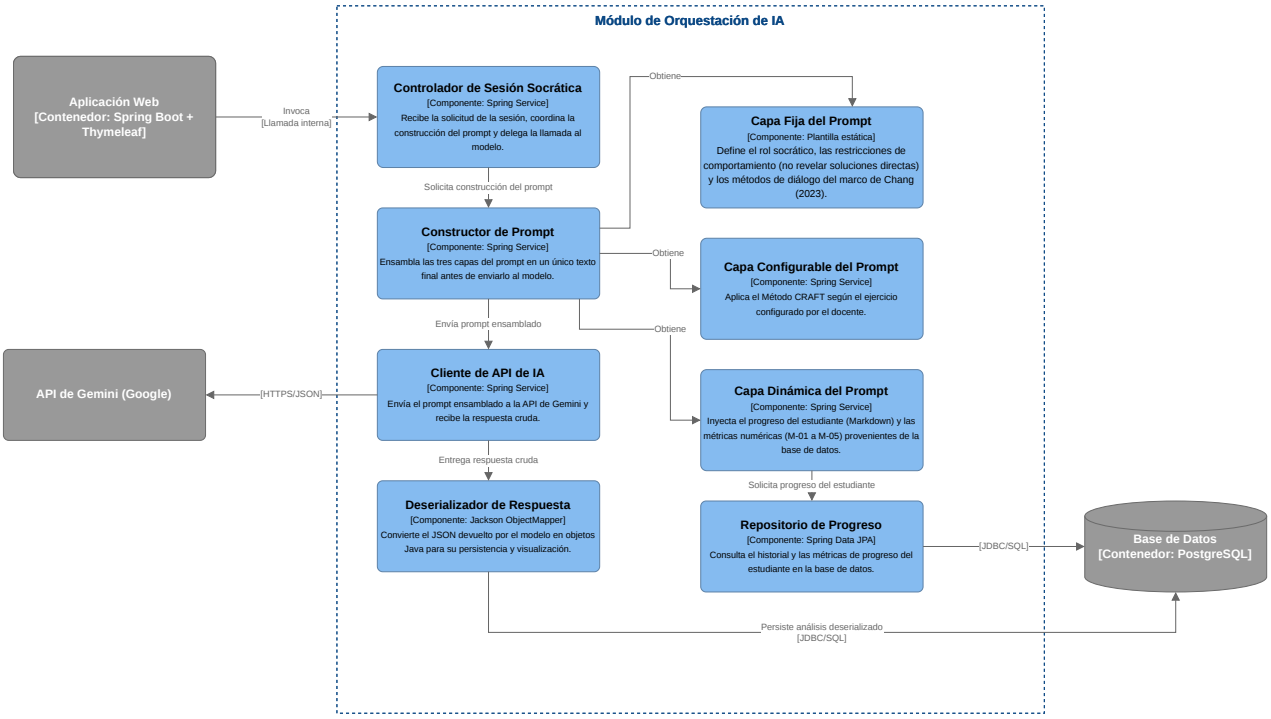


Figura 3.6: Diagrama de Componentes

Diagrama de Código

3.5.4. Esquema de Base de Datos

Como se observa en la figura (3.7), el esquema se organiza en torno a dos ejes principales, el primero es la identidad de los estudiante y el segundo es el seguimiento de su interacción dentro de cada sesión. En cuanto a las tablas de profesor y estudiante, cada una almacena la identidad de los mismos de forma independiente, evitando la duplicación de datos como nombre o correo entre distintas sesiones. Por otra parte, la relación entre una persona y una sesion se resuelve mediante la tabla participacion, tabla que además concentra el progreso cualitativo del estudiante **progreso_md** para esa sesión específica, aislándolo del resto de sus participaciones. A partir de cada participacion se derivan los componentes mensaje, que reconstruye el historial de la conversación diferenciando entre estudiante y modelo. Por último, también se encuentra la métrica, que registra en columnas estructuradas los valores numéricos y booleanos correspondientes a los cinco indicadores de seguimiento (M-01 a M-05) definidos para la plataforma.

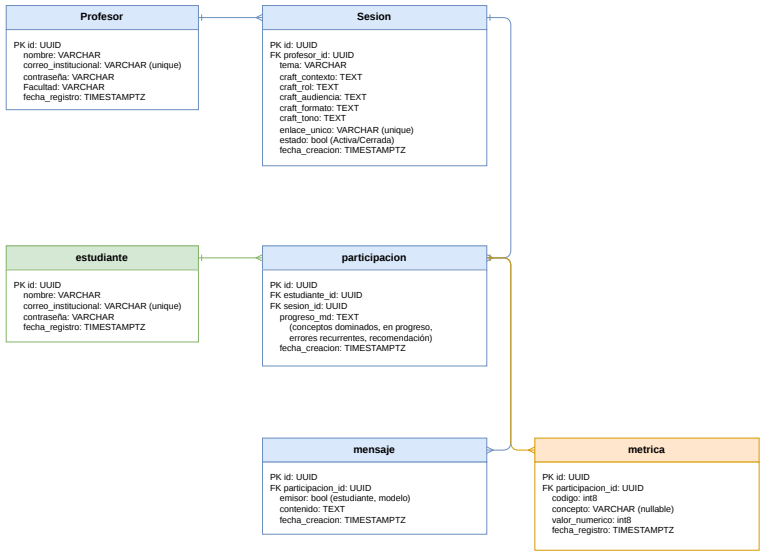


Figura 3.7: Base de Datos

3.6. Decisiones de Arquitectura y Desarrollo del Prototipo de la Plataforma

3.6.1. Arquitectura y Capa Web: Spring MVC y Spring Boot Starter Web

1. **Contexto e integración del módulo Spring MVC en la plataforma:**
   Spring MVC constituye el marco de trabajo sobre el cual se articula la arquitectura web del prototipo, fundamentado en el patrón Modelo-Vista-Controlador (MVC) y en el patrón estructural de Controlador Frontal **Front Controller** (Prasad Reddy, 2017). En el flujo del sistema de tutoría inteligente, el componente **DispatcherServlet** actúa como el punto de entrada centralizado que intercepta todas las peticiones HTTP entrantes originadas tanto por estudiantes por ejemplo en la interacción con el chat socrático, como por docentes por ejemplo en la gestión de sesiones y monitoreo de métricas (Prasad Reddy, 2017). Este componente delega la solicitud a los controladores correspondientes identificados con la anotación **@Controller**, los cuales orquestan la lógica de negocio comunicándose con la API de Gemini 3 Flash para la generación del diálogo y con Supabase para la persistencia del progreso para finalmente coordinar con el **ViewResolver** el renderizado de la interfaz final que se retorna al usuario (Prasad Reddy, 2017).

2. **Simplificación e infraestructura mediante spring-boot-starter-web:**
   El despliegue e integración tradicional de Spring MVC exige la declaración y configuración manual de múltiples bloques funcionales, tales como la asignación explícita del **DispatcherServlet**, la resolución de plantillas y el manejo global de excepciones (Prasad Reddy, 2017). Con el fin de optimizar el desarrollo de la plataforma y aplicar el principio de "convención sobre configuración", el proyecto incorpora el módulo encapsulado **spring-boot-starter-web**, el cual autoconfigura la infraestructura web de la aplicación (Prasad Reddy, 2017) a través de los siguientes elementos clave:

   - **Servidor Web Embebido y Contenerización:** Incluye un contenedor de servlets Apache Tomcat integrado por defecto (que responde convencionalmente en el puerto 8080) (Prasad Reddy, 2017). Esto permite empaquetar la plataforma como un archivo ejecutable independiente (`.jar`), facilitando su posterior despliegue y aislamiento en un entorno de contenedores Docker sin requerir la instalación ni configuración de un servidor de aplicaciones externo (Prasad Reddy, 2017).
   - **Autoconfiguración del Flujo MVC:** Inicializa automáticamente el **DispatcherServlet** mapeado a la raíz del sistema (`/`), gestiona de forma transparente la entrega de recursos web estáticos (estilos CSS, scripts JS y recursos visuales ubicados en `/static`) y activa los controladores centralizados para la captura e interpretación de excepciones durante las solicitudes de la plataforma (Prasad Reddy, 2017).
   - **Soporte Nativo para Renderizado Server-Side (Thymeleaf):** Proporciona la infraestructura preconfigurada para la vinculación directa con el motor de plantillas **Thymeleaf** (Prasad Reddy, 2017). Esto simplifica la inyección dinámica de datos contextua-

les desde la capa de servicios tales como las respuestas del tutor socrático o los historiales de conversación hacia los documentos HTML antes de ser entregados al navegador del estudiante.

- **Exposición de Servicios y APIs REST:** Facilita la construcción de controladores especializados mediante la anotación **@RestController**, aprovechando los convertidores de mensajes HTTP (`HttpMessageConverters`) para serializar y deserializar automáticamente objetos Java a formatos de intercambio como JSON (Prasad Reddy, 2017). Esta capacidad resulta indispensable para la comunicación asíncrona entre el backend de Spring Boot y servicios externos como la API de Gemini 3 Flash o la plataforma Supabase.

Capítulo 4

Conclusiones y Trabajos Futuros

4.1. Conclusiones

4.2. Trabajos Futuros

A partir del desarrollo del prototipo y de las pruebas realizadas, se identifican diversas líneas de trabajo futuro que permitirían ampliar el alcance del sistema y mejorar su robustez, precisión y aplicabilidad en contextos académicos más amplios.

En primer lugar, una línea de trabajo futuro relevante corresponde a la implementación de secuencias de pistas progresivas, en las que el sistema comience con orientaciones generales que lleven al estudiante a reflexionar por sí mismo, y escale gradualmente hacia pistas más específicas únicamente si el estudiante no logra avanzar, en línea con el principio socrático de guiar sin resolver (Colby, 2017). Si bien esta funcionalidad se consideró durante la fase de diseño, su implementación fue delimitada como trabajo futuro debido a que el comportamiento de un modelo de lenguaje no es completamente determinístico, lo que dificulta garantizar de manera consistente el nivel de progresividad de cada pista generada. Abordar esta limitación de forma robusta requeriría el desarrollo de un agente especializado o el entrenamiento de un modelo propio orientado específicamente a la generación y escalamiento de pistas, en lugar de depender únicamente de instrucciones dentro del prompt.

De manera similar, aunque la literatura sugiere que un sistema de tutoría inteligente robusto debería incorporar un modelo de seguimiento del estudiante como el Rastreo de Conocimiento Bayesiano (BKT), que permite inferir de forma continua qué habilidades ha dominado el alumno a partir de sus interacciones y sugerir la siguiente acción más apropiada cuando el estudiante se encuentra bloqueado (Kurup et al., 2016), el presente prototipo no implementa este enfoque probabilístico. En su lugar, el seguimiento del progreso del estudiante se realiza mediante un registro descriptivo en formato Markdown, generado por el modelo de lenguaje al cierre de cada sesión, lo cual permite una primera aproximación al seguimiento individual sin requerir la complejidad estadística ni los datos históricos necesarios para entrenar un modelo bayesiano confiable. La incorporación de un modelo de tipo BKT, o de un mecanismo de inferencia probabilística equivalente, se plantea como una línea de trabajo futuro que permitiría transitar de un seguimiento descriptivo a uno predictivo del dominio del estudiante.

En esa misma línea, la implementación de andamiaje (**scaffolding**) dinámico, es decir, la capacidad del sistema de proporcionar apoyo intensivo al estudiante en las etapas iniciales de un tema y retirarlo progresivamente a medida que este gana competencia, ajustando la dificultad de las

tareas a su zona de desarrollo próximo (Colby, 2017) queda igualmente delimitada como trabajo futuro. Su implementación efectiva depende, en gran medida, de contar primero con un modelo de seguimiento del estudiante más preciso que el registro en Markdown utilizado actualmente, por lo que esta línea se considera una extensión natural una vez incorporado el modelado bayesiano o un mecanismo de inferencia equivalente.

Bibliografía

Abiodun, E. F., & Isong, B. (2025). A Systematic Review of Challenges in Teaching and Learning Computer Programming Modules. *The Indonesian Journal of Computer Science*, *14*(1). https://doi.org/10.33022/ijcs.v14i1.4592

Almarales, M., & Fernández, O. (2019). Tutorial Interactivo para la Enseñanza de la Programación Orientada a Objetos. *Revista de Investigación , Formación y Desarrollo: Generando Productividad Institucional*, *7*, 8. https://doi.org/10.34070/rif.v7i3.172

Artificial Analysis. (2026). AI Model Comparison: Gemini 3 Flash reasoning, Grok 4 fast reasoning, GPT-5 mini, and Claude 4.5 Haiku reasoning [[En línea]. Disponible en: https://artificialanalysis.ai/models?models=gemini-3-flash-reasoning%2Cgrok-4-fast-reasoning%2Cgpt-5-mini%2Cclaude-4-5-haiku-reasoning. [Accedido: 30-ago-2026]].

Brown, S. (2022). *The C4 Model*. O'Reilly Media.

Castellanos, H., Vieira, C., & Magana, A. J. (2026). Feedback Systems on Computer Programming Courses: A Systematic Literature Review. *IEEE Access*, *14*, 30654-30674. https://doi.org/10.1109/ACCESS.2026.3665047

Chang, E. Y. (2023). Prompting Large Language Models With the Socratic Method. *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, 0351-0360. https://doi.org/10.1109/CCWC57344.2023.10099179

Colby, B. R. (2017). A Comparative Literature Review of Intelligent Tutoring Systems from 1992-2015. https://api.semanticscholar.org/CorpusID:150155560

Craig, J., Bleiberg, D., & Chugh, M. (2024, agosto). Implementing advanced prompt engineering with Amazon Bedrock [[En línea]. Disponible en: https://aws.amazon.com/es/blogs/machine-learning/implementing-advanced-prompt-engineering-with-amazon-bedrock/. [Accedido: 23-ago-2026]].

Deinum, M. (2018). Spring Boot—Introduction. En *Spring Boot 2 Recipes: A Problem-Solution Approach* (pp. 1-14). Apress. https://doi.org/10.1007/978-1-4842-3963-6_1

Evans, C. (2013). Making sense of assessment feedback in higher education. *Review of Educational Research*, *83*(1), 70-120. https://doi.org/10.3102/0034654312474350

Farieta-Barrera, A., & Murillo, A. (2023, enero). El método socrático y la posibilidad de una pedagogía socrática. https://doi.org/10.28970/9789585498921.01

González-Bravo, L., & Valdivia-Peralta, M. (2015). Possibilities for Using TAM and Technology Frames Models to Assess the Acceptance of New Technologies in the Chilean Higher Education Quality Assurance. *Revista Electrónica Educare*, *19*(2), 181-196. https://doi.org/10.15359/ree.19-2.11

Gusukuma, L., Bart, A. C., Kafura, D., & Ernst, J. (2018). Misconception-Driven Feedback: Results from an Experimental Study. *Proceedings of the 2018 ACM Conference on International Computing Education Research*, 160-168. https://doi.org/10.1145/3230977.3231002

IEEE Computer Society. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

Inc., D. (2026). Docker Overview [En línea. Disponible en: https://docs.docker.com/get-started/docker-overview/. Accedido: 06-jul-2026].

Inc., S. (2026). Supabase Documentation [En línea. Disponible en: https://supabase.com/docs. Accedido: 10-jun-2026].

International Organization for Standardization. (2022). Information security, cybersecurity and privacy protection — Information security management systems — Requirements (ISO/IEC Standard No. 27001:2022) [Accedido: 18-mar-2026]. https://www.iso.org/standard/27001

The Java TM Language : An Overview. (2009). https://api.semanticscholar.org/CorpusID:14097506

Joshi, D., Jocius, R., Albert, J., Joswick, C., & Blanton, M. (2026). CRAFT Prompt Generation Framework for Teachers. *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2*, 1383-1384. https://doi.org/10.1145/3770761.3777281

Kurup, L. D., Joshi, A., & Shekhokar, N. (2016). A review on student modeling approaches in ITS. *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, 2513-2517.

Microsoft Learn. (2026). Object-Oriented programming (C#) [En línea. Disponible en: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop. Accedido: 20-jun-2026].

Molina, M. (2022). What is an intelligent system? [Version 3, updated December 2022].

Monett, D., & Lewis, C. W. P. (2018). Getting Clarity by Defining Artificial Intelligence—A Survey. En V. C. Muller (Ed.), *Philosophy and Theory of Artificial Intelligence 2017* (pp. 212-214). Springer International Publishing.

Pardos, Z. A., Tang, M., Anastasopoulos, I., Sheel, S. K., & Zhang, E. (2023). OATutor: An Open-source Adaptive Tutoring System and Curated Content Library for Learning Sciences Research [En línea. Disponible en: https://doi.org/10.1145/3544548.3581574. Accedido: 26-feb-2026]. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 1-17. https://doi.org/10.1145/3544548.3581574

Pham, T., & Ngoc, T. N. (2025). Improving LLM's Response with a New Prompt Framework RC-FOR. *2025 10th International Conference on Applying New Technology in Green Buildings (ATiGB)*, 666-670. https://doi.org/10.1109/ATiGB66719.2025.11142192

Picolo, L. F., Dan, L. F., Simoes, I., & Pereira, H. G. d. S. (2025). Avaliação somativa de usabilidade: uma abordagem utilizando o System Usability Scale (SUS). *Revista Eletrônica de Iniciação Científica em Computação*, *23*(1), 228-234. https://doi.org/10.5753/reic.2025.5313

Prasad Reddy, K. S. (2017). Web Applications with Spring Boot. En *Beginning Spring Boot 2: Applications and Microservices with the Spring Framework* (pp. 107-132). Apress. https://doi.org/10.1007/978-1-4842-2931-6_10

Quiñonez, A. G., et al. (2023). Educational Platforms: Digital Tools for the teaching-learning process in Education [En línea. Disponible en: https://edsociety.iberojournals.com/index.php/IBEROEDS/article/view/626. Accedido: 26-feb-2026]. *Ibero-American Journal of Education & Society*, *3*(1), 259-263.

Rodríguez Chávez, M. H. (2021). Sistemas de tutoría inteligente y su aplicación en la educación superior. *RIDE Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, *11*(22). https://doi.org/10.23913/ride.v11i22.848

Services, A. W. (s.f.). What is an API (Application Programming Interface)? [https://aws.amazon.com/what-is/api/. Accedido: 7-mzo-2026].

Shekar, S. C., Sowmya, U., Suharshitha, P., Sahith, S. V. D., & Varun, K. S. (2026). A Smart Code Review Framework using GPT and NLP-Driven Automatic Comment Generation. *2026 Fourth International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*, 297-302. https://doi.org/10.1109/ICAISS68683.2026.11526109

Xu, S., Wen, H.-N., Pan, H., Dominguez, D., Hu, D., & Zhang, X. (2025). Classroom Simulacra: Building Contextual Student Generative Agents in Online Education for Learning Behavioral Simulation. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3706598.3713773

Zhao, J., et al. (2025). CodeEdu: A Multi-Agent Collaborative Platform for Personalized Coding Education [En línea. Disponible en: https://arxiv.org/abs/2507.13814. Accedido: 27-feb-2026]. *arXiv preprint arXiv:2507.13814*.

Zhou, L. (2025). A Review of Educational Agents: Definitions, Features, Roles and Development Trends [En línea. Disponible en: https://bonoi.org/index.php/sief/article/view/1590. Accedido: 26-feb-2026]. *Science Insights Education Frontiers*, *28*(2), 4675-4688. https://doi.org/10.15354/sief.25.re552